



貴州大學

本科毕业论文（设计）

论文（设计）题目：基于协同过滤算法的电影推荐系统

学 院： 计算机科学与技术学院(贵州保密学院)

专 业： 计算机科学与技术

班 级： 计科 2102

学 号： 2100150303

学生姓名： 王东胜

指导教师：

2025 年 6 月 2 日

贵州大学本科毕业论文（设计）

诚信责任书

本人郑重声明：本人所呈交的毕业论文（设计），是在导师的指导下独立进行研究所完成。毕业论文（设计）中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。

特此声明。

论文（设计）作者签名：王东胜

日 期：2025.6.1



目录

摘要	IV
Abstract	V
第一章 绪论	1
1.1 课题背景	1
1.2 目的和意义	1
1.2.1 研究目的	1
1.2.2 研究意义	1
1.3 国内外现状	2
1.3.1 国外现状	2
1.3.2 国内现状	3
1.4 相关技术及基本理论	3
1.4.1 相似度度量	3
1.4.2 基于邻近的协同过滤	5
1.4.3 基于隐语义模型的协同过滤	7
1.4.4 基于 TF-IDF 内容推荐算法	8
1.4.5 前端技术	9
1.4.6 后端技术	9
第二章 需求分析	12
2.1 业务需求分析	12
2.2 功能性需求	13
2.3 非功能性分析	15
第三章 系统概要设计	16
3.1 系统结构设计	16
3.2 系统架构设计	16
3.3 系统模块设计	18
3.3.1 用户基本业务模块设计	18
3.3.2 离线推荐模块设计	20



3.3.3 实时推荐模块	22
3.3.4 基于内容的推荐设计	24
3.3.5 管理员总模块	25
3.4 数据库设计	26
3.4.1 数据库的逻辑设计	26
3.4.2 数据库的物理实现	27
第四章 系统详细设计	32
4.1 用户基础业务模块设计与实现	33
4.2 管理员模块设计与实现	39
4.3 离线推荐模块设计与实现	45
4.3.1 基于离线统计的推荐设计与实现	45
4.3.2 基于 ALS 协同过滤推荐设计与实现	46
4.4 实时推荐模块设计与实现	49
第五章 系统模块实现	53
5.1 基础业务模块实现	53
5.1.1 用户登录注册实现	53
5.1.2 电影搜索功能实现	54
5.1.3 演员资源功能实现	56
5.1.4 个人中心功能实现	57
5.2 离线推荐模块实现	59
5.2.1 基于统计的推荐	59
5.2.2 基于 ALS 协同过滤的推荐	60
5.3 实时推荐模块实现	61
5.3.1 实时推荐实现	61
5.3.2 混合推荐实现	63
5.4 协同过滤冷启动问题	64
5.5 管理员业务模块实现	64
5.5.1 管理员后台首页	64



5.5.2 用户管理实现	65
5.5.3 可视化分析	65
5.5.4 评论管理	66
5.5.5 电影管理	66
第六章 系统测试	68
6.1 功能测试	68
6.1.1 用户功能测试	68
6.1.2 管理员功能测试	71
6.2 非功能性测试	73
第七章 总结与展望	74
7.1 总结	74
7.2 展望	74
参考文献	75
致谢	76



基于协同过滤算法的电影推荐系统

摘要

信息技术发展致海量数据使信息筛选低效、内容同质化，电影领域作品激增与用户个性化需求矛盾突出。传统搜索与推荐方式有局限，用户找满意电影耗时低效。在此背景下，个性化推荐孕育而生。因此本文结合多种推荐算法混合推荐完成推荐算法之间的互补。本文主要完成如下工作：

第一，本文针对于用户角色设计了普通用户业务系统以及管理员的后台管理系统。本系统设计了离线推荐和实时推荐功能。其中，离线推荐功能包括两类：一类是基于统计的推荐，包含热门推荐与评分数量最多推荐；另一类是基于 ALS 协同过滤算法的推荐，分别从用户维度和电影维度进行离线推荐，挖掘用户与电影潜在关联。实时推荐是引入了调节因子改良 ALS 协同过滤来强化用户近期行为数据的影响，让电影推荐在用户评分后能进行实时更新提高用户体验。此外，为有效解决冷启动问题，本文引入了基于 TF-IDF 的内容推荐方法，和上述推荐策略相结合，实现混合推荐，进一步提升推荐的精准度与全面性。

第二，本系统前端采用 Vue 框架进行开发，后端业务使用 SpringBoot 框架进行开发，引入 Spark 生态来完成系统的相关计算。最终搭建一个高稳定和高性能的推荐系统。

关键词：协同过滤,混合推荐，Spark，ALS



Movie Recommendation System Based on Collaborative Filtering Algorithm

Abstract

The development of information technology has led to an explosion of data, resulting in inefficient information filtering and content homogenization. In the film industry, the surge in the number of works stands in stark contrast to users' personalized demands. Traditional search and recommendation methods have limitations, making it time-consuming and inefficient for users to find satisfactory movies. Against this backdrop, personalized recommendation systems have emerged. Therefore, this paper proposes a hybrid recommendation approach that combines multiple recommendation algorithms to achieve complementarity. The main contributions of this paper are as follows:

Firstly, this paper designs a business system for regular users and a backend management system for administrators, targeting different user roles. The system incorporates both offline and real-time recommendation capabilities. Offline recommendations are divided into two categories: one is statistics-based, including popular recommendations and recommendations based on the highest number of ratings; the other is based on the ALS (Alternating Least Squares) collaborative filtering algorithm, which conducts offline recommendations from both user and movie dimensions to uncover latent associations between users and movies. Real-time recommendation introduces an adjustment factor to improve the ALS collaborative filtering algorithm, thereby enhancing the influence of recent user behavior data. This enables movie recommendations to be updated in real-time after user ratings, enhancing the user experience. Furthermore, to effectively address the cold-start problem, this paper introduces a TF-IDF-based content recommendation method, which is combined with the aforementioned recommendation strategies to implement a hybrid recommendation approach, further improving the accuracy and comprehensiveness of recommendations.



Secondly, the front-end of this system is developed using the Vue framework, while the back-end business logic is implemented using the SpringBoot framework. The Spark ecosystem is integrated to handle relevant computations within the system. Ultimately, a highly stable and high-performance recommendation system is established.

Keywords: Collaborative Filtering, Hybrid Recommendation, Spark, ALS



第一章 绪论

1.1 课题背景

随着信息技术的快速发展和互联网的普及，海量数据导致用户在信息筛选过程中面临效率低下、内容同质化等问题。尤其在电影领域，作品数量的爆炸式增长与用户个性化需求之间的矛盾日益凸显。传统搜索引擎依赖用户主动输入明确需求，而用户很多时候往往难以清晰描述兴趣偏好；传统推荐方式（如热门推荐）缺乏针对性，难以满足用户的差异化需求。用户想要找到自己满意的电影会有耗时高，效率低的痛点。因此，本文设计提出基于协同过滤算法的推荐系统来解决信息过载问题的关键技术，通过分析用户显式行为（评分、收藏）和隐式行为（点击、停留时长）挖掘潜在兴趣，推动用户从“主动搜索”向“智能推荐”的体验转变，为用户提供个性化推荐的服务。

1.2 目的和意义

1.2.1 研究目的

本研究旨在构建一套基于协同过滤算法的个性化电影推荐系统，解决传统推荐模式下用户面临的信息冗余、同质化推荐等问题。通过整合用户显式行为（如评分、收藏）与隐式行为（如点击频次、观影时长）的多维度数据，深入挖掘用户潜在兴趣偏好，突破“热门推荐”“随机推送”等粗放式推荐逻辑，实现从“人找内容”到“内容找人”的智能化转型。同时，系统设计注重提升平台的商业运营效能，将用户行为数据转化为可落地的运营策略，为电影资源的精准投放、用户留存率提升及消费转化率优化提供技术支持，探索算法模型在文化娱乐产业中的实践价值。

1.2.2 研究意义

在用户价值层面，个性化推荐系统通过分析用户行为特征，能够主动发现其尚未明确表达的小众化观影需求，帮助用户在海量内容中快速定位兴趣点，有效缓解信息过载引发的决策疲劳。这种“越用越懂你”的推荐机制，不仅缩短了用户从搜索到观影的行为路径，更通过持续优化的推荐结果构建起“观看-反馈-再推荐”的良性互动闭环，显著增强用户对平台的使用粘性与满意度。

从商业发展视角来看，推荐系统的精准度直接关联平台的核心竞争力。本系统通过提升电影与用户兴趣的匹配效率，可缩短“兴趣激发-消费决策”的转化周期，促进会员



订阅、单片点播等盈利增长点。此外，系统沉淀的用户偏好数据能够反向指导平台的内容采购、版权合作等战略决策，形成“数据积累赋能业务优化，业务优化反哺数据价值”的可持续发展模式。

就行业创新而言，本研究不仅验证了协同过滤算法在电影推荐场景中的技术可行性，更为文化娱乐产业的智能化升级提供了可迁移的实践范式。推荐系统作为连接内容生产者与消费者的数字纽带，可推动电影产业从依赖主观经验判断的资源密集型模式，转向以数据驱动决策的精准化运营模式。这种转型不仅能够降低内容分发成本，更可通过用户偏好分析反哺上游创作环节，为行业生态的协同进化注入新动能。

1.3 国内外现状

1.3.1 国外现状

当代推荐系统起源于上个世纪 90 年代。在过去的几十年里，推荐系统已经得到了广泛的应用和发展，已经成为现代电子商务、社交网络和在线媒体等领域的重要组成部分^[1]。此后，来自不同领域的各大学者敏锐洞察先机，纷纷投身于推荐算法这一新兴领域的研究探索之中。

1994 年，GroupLens 系统开始给人们推荐文章，这是设计的首个主动为用户推荐文章的系统，自此之后推荐系统开始作为一个独立的研究方向受到研究学者们的重点关注^[2]。主动为用户推荐的提出标志着推荐系统领域的重要突破，它首次通过群体智慧解决信息过载问题，利用用户行为相似性而非内容分析实现个性化推荐，为后续的推荐算法研究与应用奠定了核心范式基础。随着推荐算法的优化和个性化推荐的特点随后很多公司将该推荐算法应用起来并取得了良好的效果。2007 年美国计算机学会（ACM）举办了首届推荐系统大会（ACM recsys），会上集中发表了大量推荐系统领域优秀研究成果和论文，作为推荐领域最高水平会议，目前已举办十多次，在推荐系统的发展中发挥了重要作用^[3]。随后 2010 年，YouTube 提出了一种改进的协同过滤算法，他们将视频进行关键词提取使用关键词作为特征向量与用户的行为特征相结合，最后计算出与用户喜欢的视频相似的视频，然后将相似度高的进行推荐。进入 21 世纪，电影推荐系统迎来了矩阵分解^[4]和逻辑回归这两种新技术。Netflix 公司实际上也一直在深入研究电影推荐系统，其公司首席产品官声称全球约 80%或以上的视频用户选择的电影与视频均来自



个性化的推荐系统，并公开宣称由 Netflix 开发设计的推荐系统，每年给公司带来的收益将达到十亿美元之多^[5]。

1.3.2 国内现状

随着信息技术的迅猛发展，尤其是互联网和移动互联网的普及，国内每天产生的信息量呈指数级增长。同时人们日益增长的文化追求，电影作为一种重要的文化娱乐形式，其数量也在不断膨胀。因此对于商品、电影等物品的精准推荐也成为了国内的挑战。

国内在推荐领域的研究起步较晚，其研究主要集中于针对协同过滤、关联规则以及矩阵分解算法的局限性，对其进行改良，进而提升推荐算法的推荐质量^[6]。2009 年，我国百分点信息技术有限公司率先推出自主研发的个性化推荐系统，针对国内电商市场特点，为京东、当当等头部平台量身打造动态用户建模与智能匹配引擎，开创性地解决了大规模商品库下的精准推荐难题，推动国内电商推荐技术迈入商业化应用新阶段。随后 2021 年百度系应用通过深度学习模型实时解析用户兴趣图谱，结合百亿级搜索行为数据构建动态用户画像，不仅实现内容生态与用户需求的无缝对接，更通过预测性广告匹配技术，将商业信息精准触达高转化潜力用户，形成"兴趣感知-行为预测-价值转化"的闭环营销链路。随后，腾讯视频、优酷、爱奇艺等多家视频播放平台纷纷上线个性化推荐功能，通过深度解析用户观影偏好与行为轨迹，实现内容分发的精准匹配，不仅显著提升了用户粘性与观看时长，更有效拉动了会员订阅增长和影片购买转化率，开创了内容消费与用户体验双赢的新局面。

1.4 相关技术及基本理论

本小节主要介绍本次实现系统中所用到的具体技术以及相关的推荐算法。对于系统的实现采用了前后端分离的架构以及数据处理部分使用 Spark 相关的大数据生态完成系统计算工作。推荐算法则是采用基于 ALS 的协同过滤算法完成基于用户和基于电影的推荐。同时使用改良因子改进的隐语义模型来实现实时推荐。最后为了解决冷启动问题，本文接住了 TF-IDF 算法完成基于内容推荐。

1.4.1 相似度度量

相似性度量作为机器学习领域的核心理论基础，在推荐系统与聚类分析中发挥着关键作用。其核心功能在于量化数据对象间的关联强度，直接决定了推荐算法中用户兴趣

建模的精准度及聚类算法中群体结构划分的合理性。研究表明，相似性度量的选择策略是提升推荐效果、优化用户体验的重要突破口^[7]。当前主流方法包括：余弦相似度（适用于文本等稀疏向量场景）、皮尔逊相关系数（能消除用户评分偏差的干扰）以及欧氏距离（反映特征空间中的绝对距离差异），这些方法在不同数据特性下展现出差异化的优势。通过科学选择相似性度量，可显著提升算法在稀疏数据环境下的鲁棒性及高维空间中的计算效率。

（1）欧几里得距离

欧几里得距离，又称欧氏距离，是一种基础且易于理解的相似性度量。在多维空间中，它用于衡量两点间的绝对直线距离，两点间欧几里得距离越小，表明二者越相近。然而，该距离度量方法存在局限性，因其对事物不同属性差异一视同仁，难以契合复杂多变的实际需求，故在事物度量标准统一、各属性权重等同的场景中应用效果更佳。欧几里得距离计算公式如公式（2.1）所示：

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2.1)$$

（2）皮尔逊相关系数

皮尔逊相关系数是衡量两个变量线性相关程度的指标，取值范围在 -1 到 1 之间。值为 1 表示完全正相关，-1 表示完全负相关，0 表示无线性相关。它基于协方差和标准差计算，能直观反映变量间线性关联的紧密程度与方向。皮尔逊相关系数计算公式如公式（2.2）所示：

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (2.2)$$

（3）余弦相似度

余弦相似度是通过两个向量之间的夹角的余弦值来判断相似程度的。范围在-1 到 1 之间。 $\cos \theta$ 值为 0 表示两个向量毫不相干，接近 1 相似性越高，接近-1 说明两个向量方向相反。余弦相似度计算公式如公式（2.3）所示：

$$\cos \theta = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (2.3)$$

1.4.2 基于邻近的协同过滤

核心思想：通过挖掘与目标用户兴趣相似的用户群体（用户 CF）或目标物品相似的物品集合（物品 CF），借助近邻的评分行为预测目标用户对未接触物品的偏好。

（1）基于用户的协同过滤

基于用户的协同过滤算法核心在于寻找与目标用户兴趣相似的“邻居”用户群。通过计算用户间相似度，找出相似用户，再依据这些邻居对物品的评分或行为，预测目标用户对未接触物品的偏好，进而生成个性化推荐。简单来说用户协同过滤是基于相似用户的行为来推荐内容^[8]。

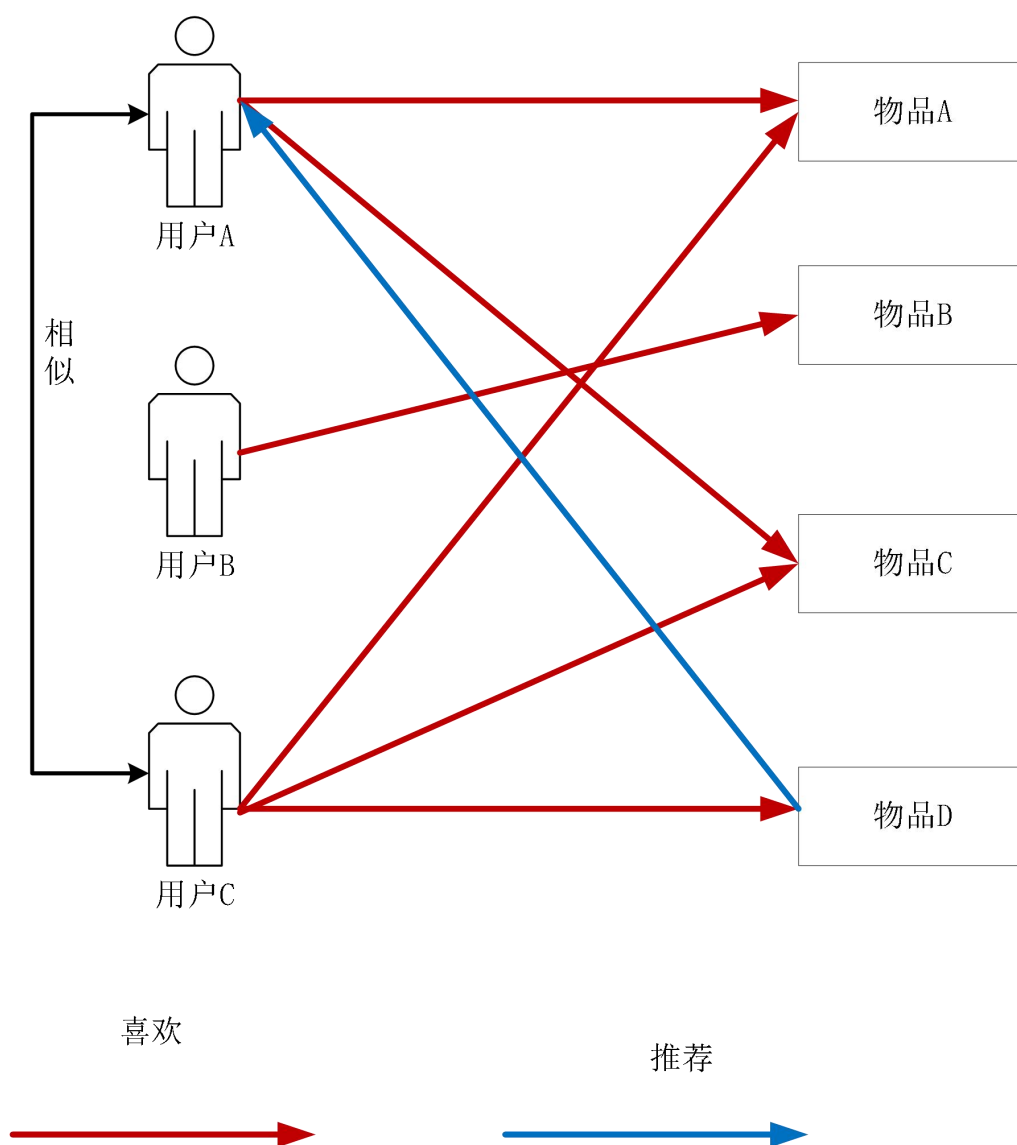


图 1.1 基于用户的协同过滤示例图

如图 1.1，根据余弦相似度算法，用户 A 和用户 C 都共同喜欢物品 A 和物品 C，

因此可以判定用户 A 和用户 C 相似度大，并且互相为邻居。最后将用户 C 喜欢而用户 A 没接触过的物品 D 推荐给用户 A

（2）基于物品的协同过滤

基本原理和基于用户的类似，基于物品的协同过滤，通过计算物品间相似度，根据用户历史偏好物品的相似物品进行推荐。

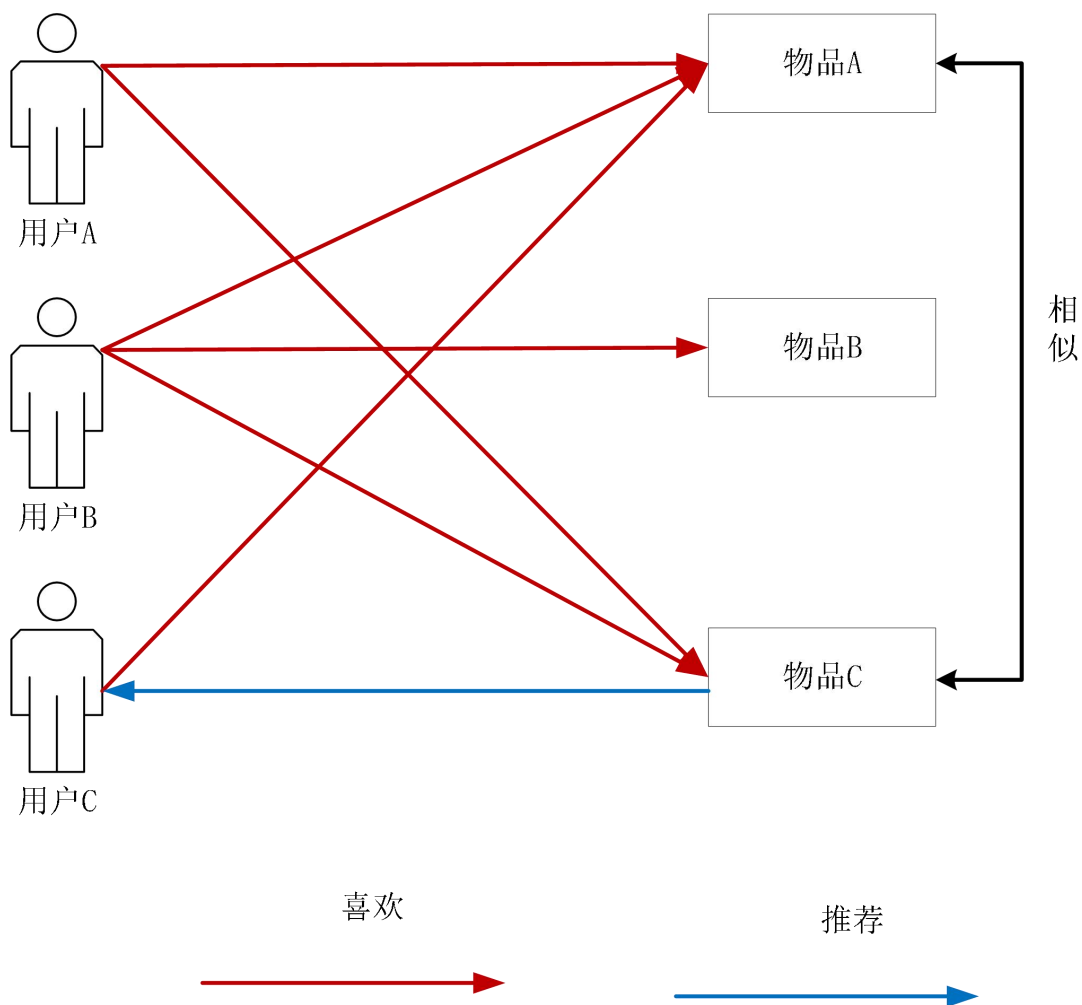


图 1.2 基于物品的协同过滤示例图

如图 1.2，根据余弦相似度算法，物品 A 和物品 C 都同时被用户 A 和用户 B 喜欢，因此可以判定物品 A 和物品 C 相似度较大，此时可以看到用户 C 喜欢物品 A,且没接触过 C 物品，那么就将物品 C 推荐给用户 C。

（3）基于邻近的协同过滤的优缺点

基于协同过滤的推荐机制的优点：原理直观易懂，通过计算用户或物品间的相似度，找出相似用户喜爱的物品或相似物品被哪些用户喜欢，进而进行推荐。它不需要对物品

内容进行复杂分析，适用范围广，可应用于多种领域。

基于协同过滤的推荐机制所存在的问题：首先依赖历史数据，新用户与新物品因无相关记录，冷启动问题突出。其次推荐质量受用户历史偏好数据的丰富度和准确性影响大，数据有缺或出错，推荐效果便大打折扣。最后因为评分表可能欠缺数据会导致稀疏矩阵问题，计算复杂且易被小部分错误偏好误导。

1.4.3 基于隐语义模型的协同过滤

（1）LFM 矩阵分解模型

LFM 矩阵分解模型本质是利用矩阵分解的方式得到用户与隐含用户特征、物品特征之间的关系^[9]，将用户评分矩阵 R_{ui} 分解为用户隐特征矩阵 P_{uf} 与项目隐特征矩阵 Q_{if} ，并且两矩阵乘积会尽可能拟合现有评分数据，从而实现对未评分项目的预测。LFM 模型通过公式（2.4）预测用户 u 对项目 i 的评分。其中 P 、 Q 分别为 $user$ 和 $item$ 的特征矩阵，其中 k 就是隐类的特征维度。

$$\hat{R}_{ui} = P_u^T \cdot Q_i = \sum_{k=1}^K P_{uk} \cdot Q_{ki} \quad (2.4)$$

矩阵分解得到的预测评分矩阵 $\hat{R}$ 与原来评分矩阵 R 在已知的评分项上可能会有误差，因此需要找到一个最好的分解方式，让分解之后的预测评分矩阵总误差最小。而公式（2.5）定义的损失函数就能较好的解决这个问题，其中的引入的 $\lambda \sum_u ||P_u||^2 + \lambda \sum_i ||Q_i||^2$ 为正则化项，目的是防止过度拟合。

$$C = \sum_{(u,i) \in R_0} (R_{ui} - P_u^T \cdot Q_i)^2 + \lambda \sum_u ||P_u||^2 + \lambda \sum_i ||Q_i||^2 \quad (2.5)$$

（2）基于 ALS 的协同过滤推荐算法

该算法使用了基于物品的协同过滤推荐算法，通过计算物品之间的相似度，结合用户的历史行为数据推荐于用户喜好相似的物品。ALS(Alternation Least Squares, 交替最小二乘法)完成了 LFM 矩阵分解模型中提到的将原始评分数据 R 分解为用户特征矩阵 P ，和电影特征矩阵 Q 。

ALS 的思想是：由于两个矩阵 P 和 Q 都未知，且通过矩阵乘法合在一起，为了使它们解偶，可以先固定 Q ，把 P 当作变量，通过损失函数最小化求出 P ，这就是一个经典的最小乘问题；再反过来固定求得的 P ，把 Q 当作变量，求解出 Q ；如此交替执行，直到误

差满足阈值条件，或者到达迭代上限^[10]。

首先固定 Q 矩阵，对 P 求偏导后得到公式（2.6），然后再利用式(2.6)将 P 一行一行更新，更新完毕之后在用同样的方式更新 Q ，最后解出 P ， Q 。

$$\mathbf{P}_u = (\mathbf{Q}_{k \times n} \mathbf{Q}_{k \times n}^T + \lambda \mathbf{E})^{-1} \mathbf{Q}_{k \times n} \tilde{\mathbf{r}}_{ui} \quad (2.6)$$

对于矩阵分解的效果有必要评估预测准确度以实现优化目的。最终的目标是最大程度地减少预测误差，从而提高预测的准确度。使用均方根误差(Root Mean Square Error, RMSE)作为评估标准。 $\text{RMSE} \in [0,1]$,其中 $\text{RMSE}=0$ 表示无误差， RMSE 越接近 1 表示准确度越低^[11]。其表达式如公式（2.7）所示：

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\text{observed}_i - \text{predicted}_i)^2} \quad (2.7)$$

1.4.4 基于 TF-IDF 内容推荐算法

TF-IDF (Term Frequency-Inverse Document Frequency) 是一种广泛应用于信息检索与文本挖掘的加权技术，通过统计方法评估词语在文档集中的重要性。TF-IDF 算法的主要思想是如果某个词汇在一个文本中出现的频率越高，并且在其他文本中很少出现，则认为此词汇具有很好的类别区分能力^[12]。本文主要用该算法来进行基于内容的推荐。具体算法原理如下：

（1）词频 (Term Frequency, TF)：作用为某词在文档中出现的次数与该文档总词数的比值，反映该词在文档中的重要程度。给定词语 i 在文档 j 中其中， n_{ij} 表示词语 i 在文档 j 中出现的次数， $Q_{*,j}$ 表示文档 j 的总词数，对应的词频计算公式如公式（2.8）所示：

$$\text{TF}_{i,j} = \frac{n_{ij}}{n_{*,j}} \quad (2.8)$$

（2）逆文档频率 (Inverse Document Frequency, IDF)：逆文档频率量化词语的全局区分能力。其计算公式引入对数变换以抑制高频词的权重，其中 N 表示文档总数， N_i 表示包含词语 i 的文档数，公式如（2.9）所示：

$$\text{IDF}_i = \log \left(\frac{N+1}{N_i+1} \right) \quad (2.9)$$

（3）TF-IDF 综合计算：词语的最终 TF-IDF 值为 TF 与 IDF 的乘积公式如(2.7)所示，其中计算出来的值越大表明词语对文档的区分能力越强，公式如（2.10）所示：



$$TF - IDF = TF \times IDF \quad (2.10)$$

1.4.5 前端技术

Vue 是一个用于构建用户界面的渐进式 JavaScript 框架。它采用组件化开发，将界面拆分为独立、可复用的组件，提升开发效率与代码可维护性。其核心库只关注视图层，易与其他库或已有项目整合。通过响应式数据绑定，数据变化自动更新视图。还具备虚拟 DOM 等特性，优化性能，适合开发单页面应用（SPA）及复杂交互的前端项目。

1.4.6 后端技术

由于推荐算法需要结合海量数据作为基础，通过存贮与深挖用户历史行为数据精准捕捉用户兴趣与需求以提升推荐精准度，故推荐系统须依托涵盖海量数据的存贮、处理及分析全链条技术体系的大数据技术构建高效、稳健、可扩展的推荐平台，因此本系统后端部分除了使用传统的后端处理技术还引入了相关的大数据组件对数据进行处理。具体使用框架及其技术简介如下：

（1）SpringBoot 框架

SpringBoot 旨在简化 Spring 应用的初始搭建与开发过程。它通过自动配置和起步依赖，让开发者能快速构建独立、生产级的 Spring 应用，减少配置工作，提升开发效率。在本系统中主要充后端业务处理框架。

（2）Spark 平台

Spark 作为一个新兴的大数据处理框架，以其高效、易用和通用性等特点在业界备受推崇^[13]。Apache Spark 构建的开源大数据处理平台，集成了 Spark Core、Spark SQL、Spark Streaming 和 Spark MLlib 等核心模块，打造出统一的技术栈体系，能高效支撑批处理、实时流处理、交互式查询以及机器学习建模等复杂应用场景，且凭借标准化接口可与 Hadoop 生态组件及云计算平台无缝集成；在架构层面，该平台通常被划分为基础引擎层、数据处理层、分析计算层和交互接口层这四个层次。具体结构图如图 1.3 所示：

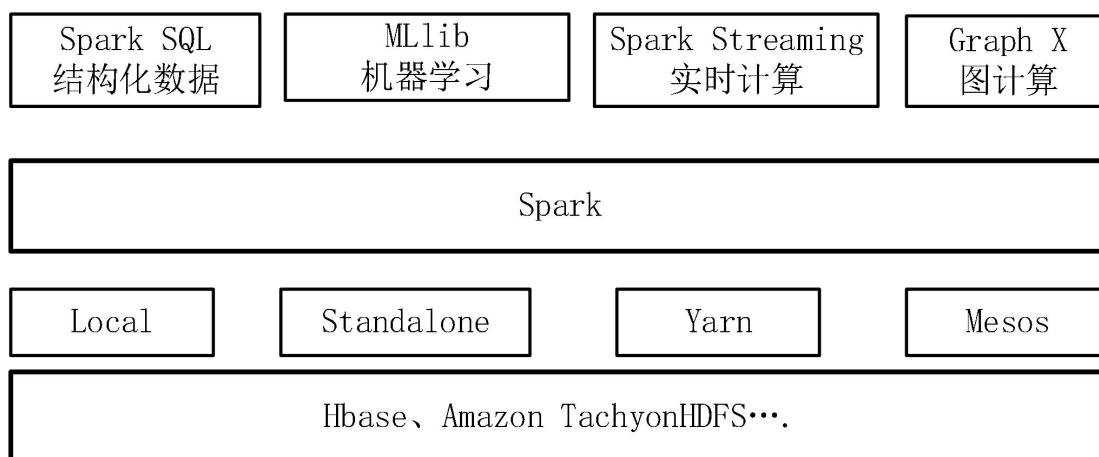


图 1.3 Spark 生态结构图

本文系统中使用到了 Spark SQL、Spark Streaming 和 Spark MLlib 因此在这简单介绍一些对应的功能：

1) Spark SQL 作为基于内存计算的结构化数据处理引擎，通过 Catalyst 优化器支持标准 SQL 查询及 DataFrame 和 Dataset API，实现了对结构化与非结构化数据的高效处理；

2) Spark Streaming 通过微批处理架构提供端到端的实时数据管道能力，兼容 Kafka、Flume 等主流数据源，支持滑动窗口、状态管理等复杂流处理模式 Spark

3) Spark MLlib 是 Spark 的机器学习库，支持大规模数据处理，提供多种算法与工具，可高效构建机器学习模型与应用。本系统中主要利用 Spark MLlib 中的矩阵分解算法 ALS，通过交替最小二乘法实现协同过滤推荐。

（3）中间件

MongoDB 是一种非关系型（NoSQL）数据库，它采用文档型数据存储模型，数据以类似 JSON 的 BSON 格式存储。它支持灵活的数据结构，便于快速开发迭代，且具备高扩展性、高性能等优势，广泛应用于互联网、物联网等领域。在本系统中充当业务数据库。

Redis 是一个开源的高性能键值对数据库，以内存存储为主，支持多种数据结构。它读写速度快，常用于缓存、消息队列、会话存储等场景，还支持数据持久化、主从复制和集群部署，保障高可用性。

Elasticsearch 是开源的分布式搜索与分析引擎。它支持实时数据搜索和分析，能处



理大规模数据集，提供全文检索、复杂查询及聚合分析等功能。本文主要使用 Elasticsearch 搜索中的 TF-IDF 算法做基于内容的推荐。

Kafka 是 Apache 开源的分布式流处理平台与消息队列系统，由 LinkedIn 开发。它支持高吞吐量、可扩展性和容错性，能处理海量实时数据，提供消息发布与订阅功能，广泛应用于日志收集、消息队列、实时数据处理等场景。

Azkaban 是由 LinkedIn 开源的批量工作流任务调度器，用于按特定顺序运行一组任务。它支持任务依赖关系管理，提供可视化 Web 界面，方便用户创建、调度和监控任务流，还支持定时调度、身份验证等功能，适用于复杂工作流场景。



第二章 需求分析

2.1 业务需求分析

对于用户端核心业务：用户可以直接通过 web 界面直接访问系统，本系统旨在为用户提供一个个性化的电影推荐、除此之外系统为电影进行了分类，用户可以直接按照类型筛选电影、以及提供电影的模糊匹配来提高用户对于电影的主动搜索。而对于系统主动推荐部分主要业务如下：

用户端：首先如果用户未注册直接浏览的主页面，那么系统默认给用户推送热点电影、评分最高的电影等（非个性化推荐），对于新用户注册，后系统会主动收集用户的兴趣表，之后系统就可以根据用户兴趣做一个关于电影内容的推荐。如果登陆的用户有相关的行为数据，那么系统会对用户进行个性化推荐如实时推荐、基于用户的推荐、混合推荐。在用户点击电影之后系统会需要根据电影来为用户推荐相似的电影，在用户对电影打分之后，后台需要记录用户的该行为记录，然后为用户更新对应的实时推荐列表、以及混合推荐列表。除了推荐相关内容之外，系统需提供给用户查看、修改个人信息，查看、修改、删除自己的历史评论信息以及评分信息等。

管理员端：管理员登录后台系统之后，可以对整个系统相关数据进行管理来保证系统的正常运行，包括对于用户的用户信息管理、用户行为数据的管理如评论以及平台核心电影模块的数据管理。后台系统还需提供对于平台提供的推荐电影的相关转化率分析，方便管理员后续运营平台。

业务主要流程图如图 2.1 所示：

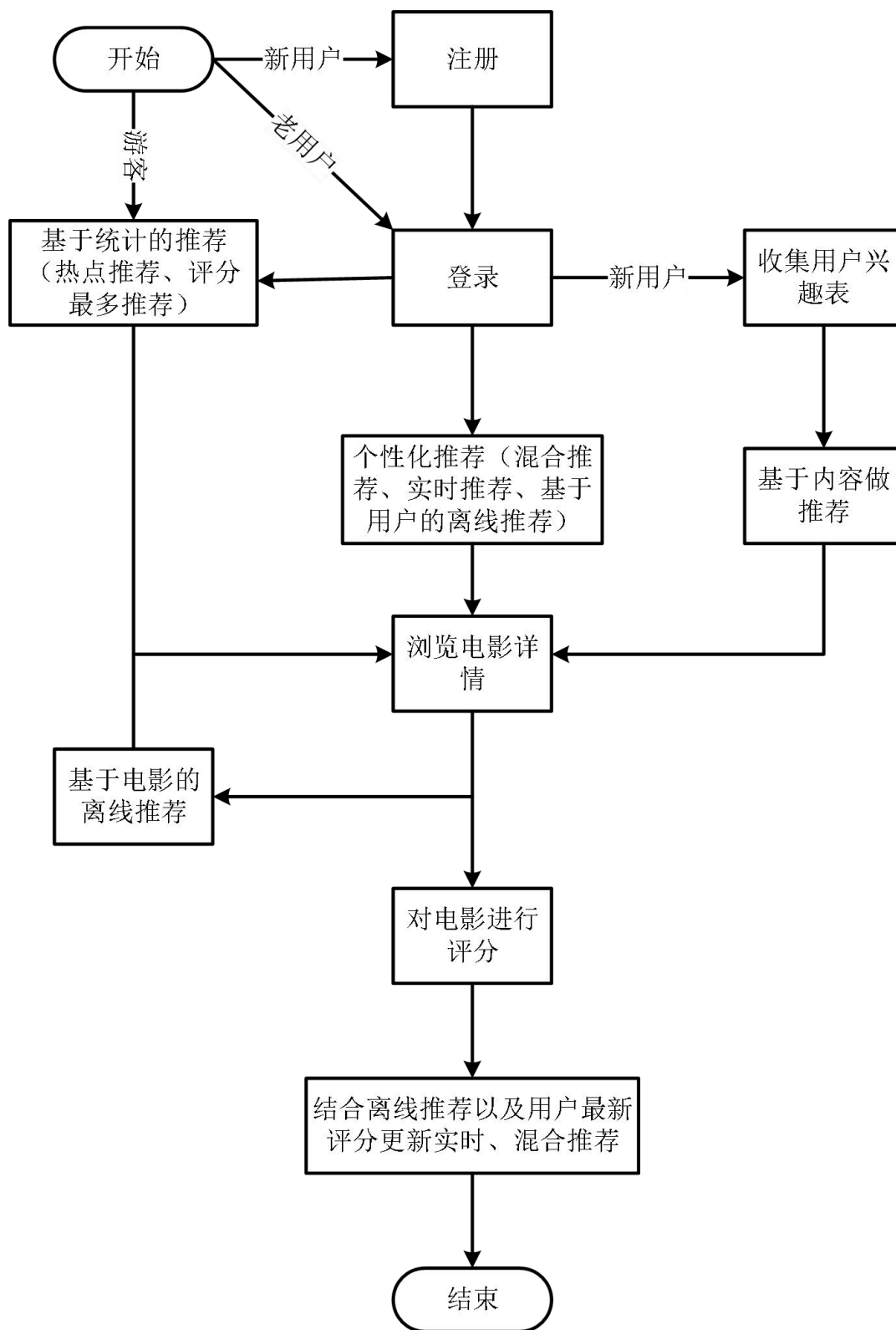


图 2.1 主要业务流程图

2.2 功能性需求

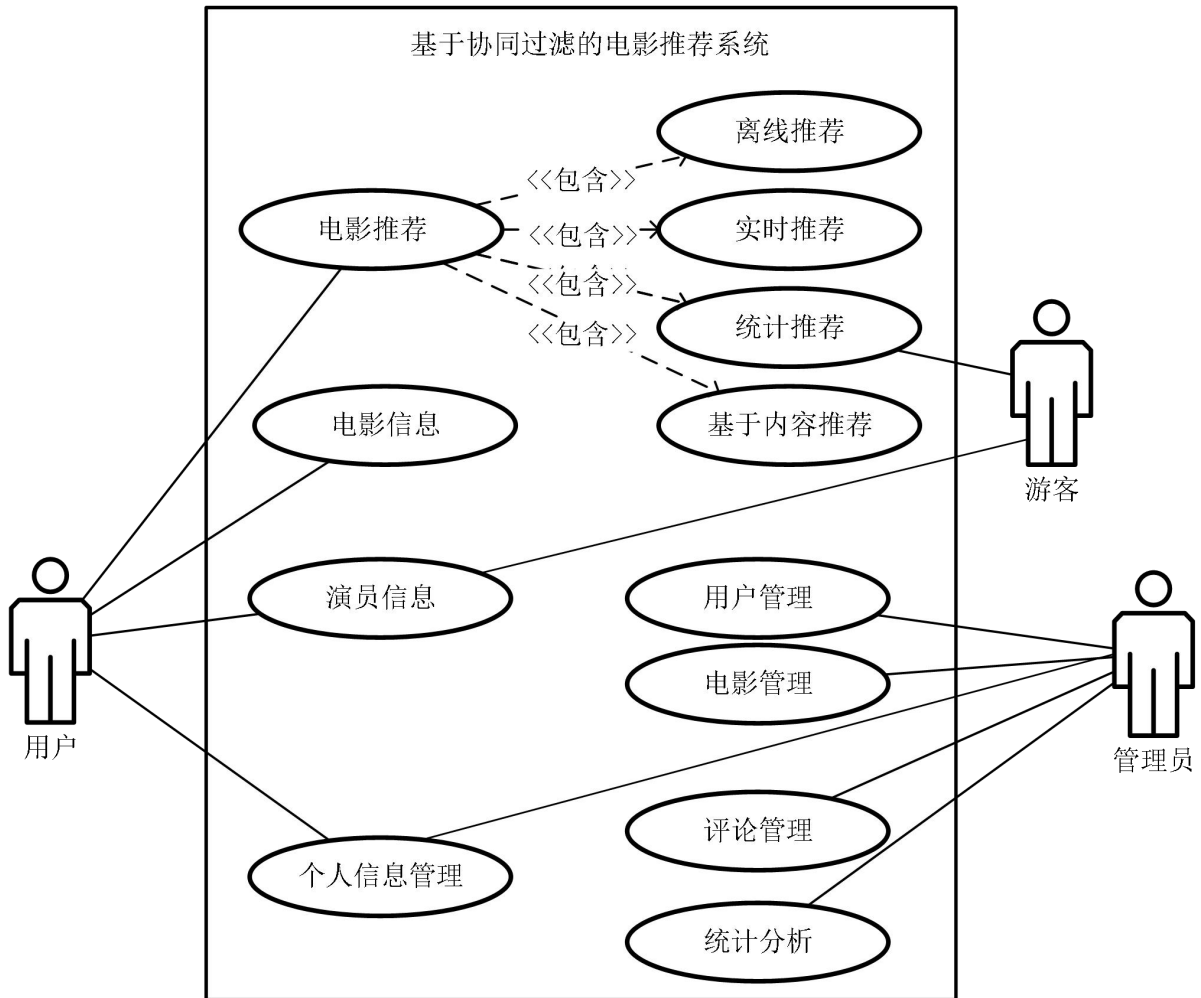


图 2.2 系统用例图

系统用例图如图 2.2 所示，用例图描述如下：

本系统使用方式主要有三种角色：用户、游客、管理员。每种角色在本系统能使用的系统功能不尽相同。详细描述如下：

（1）用户登录系统

业务功能：用户通过移动端应用/Web 端进入电影推荐系统，在用户登录之后用户可以使用系统的电影推荐、电影信息、演员信息以及个人信息管理等其中电影推荐主要包括离线推荐、实时推荐、混合推荐、基于内容的推荐等。

（2）游客角色使用系统

业务功能：这里的游客角色主要是指用户未登录时使用系统时的角色，游客角色主要可以使用系统的基于统计的电影推荐比如热点推荐、评分最多推荐以及演员信息查看等功能。



（3）管理员使用系统：

业务功能：管理员除了可以查看普通用户界面之外还增加了对应的管理员后台系统。支持用户管理、电影管理、评论管理和系统统计分析比如可视化系统推荐转化率等功能。

2.3 非功能性分析

（1）易用性：易用性体现在用户是否可以快速入手、是否容易操作，用户与界面的交互大多采用点击或文本框的方式进行交互，故而没有复杂的界面交互方式，况且界面相对简单，便于用户快速地上手并且熟悉。

（2）系统性能需求：系统应具备高性能，用户在浏览相关电影、演员相关页面时要满足时效性。

（3）安全需求：用户数据应得到保护，包括用户个人信息和代码内容，确保不被恶意获取或篡改，以及在后端返回数据给前端的过程中做到对数据的脱敏处理。

（4）可靠性需求：系统应具备高可靠性，保证用户在任何时间都能正常使用系统，且不会因系统故障而导致数据丢失或错乱。系统应有备份和恢复机制，确保在意外情况下能够及时恢复数据和功能。

（5）可维护性需求：为了方便后期项目的持续更新迭代以及改良，本系统需要实现高扩展性即后续如果还有其他功能加入或者有其他优化方案的时候能够不影响老的业务。

第三章 系统概要设计

3.1 系统结构设计

本系统主要完成基于用户的电影推荐系统以及基于管理员端的后台管理系统。用户端主要实现了与用户个人相关的登录注册、个人信息管理和用户历史行为数据如历史评论历史评分电影等和与电影推荐相关的不仅提供基于离线统计的热点推荐、评分最多推荐，还运用离线协同过滤算法，实现基于用户相似度和电影相似度的精准离线推荐。同时，结合用户实时评分反馈，动态生成实时推荐，并将多种推荐策略有机融合，提供个性化的混合推荐服务。管理员端则是完成对于用户、电影、评分的管理功能以及对于系统推荐转化率的可视化分析。对应的系统功能结构如图 3.1 所示：

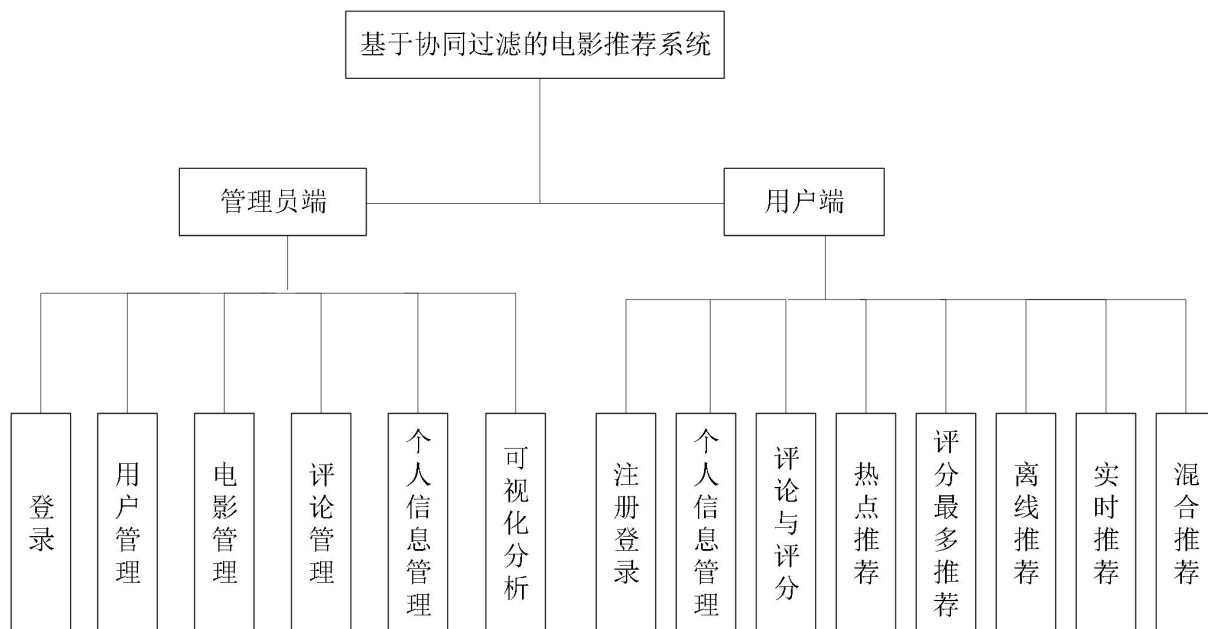


图 3.1 系统功能结构图

3.2 系统架构设计

本系统整体架构采用 B/S(浏览器/服务器)三层架构，即表示层、业务层、数据访问层，在这种架构下，项目管理信息系统能实现定制化的修改、改进和增强，创建新的前端应用控制软件，最大限度地实现前端软件的可重用性和可扩展，增强前端兼容性^[14]。并且降低了框架之间的耦合度。本系统的架构如图 3.2 所示：

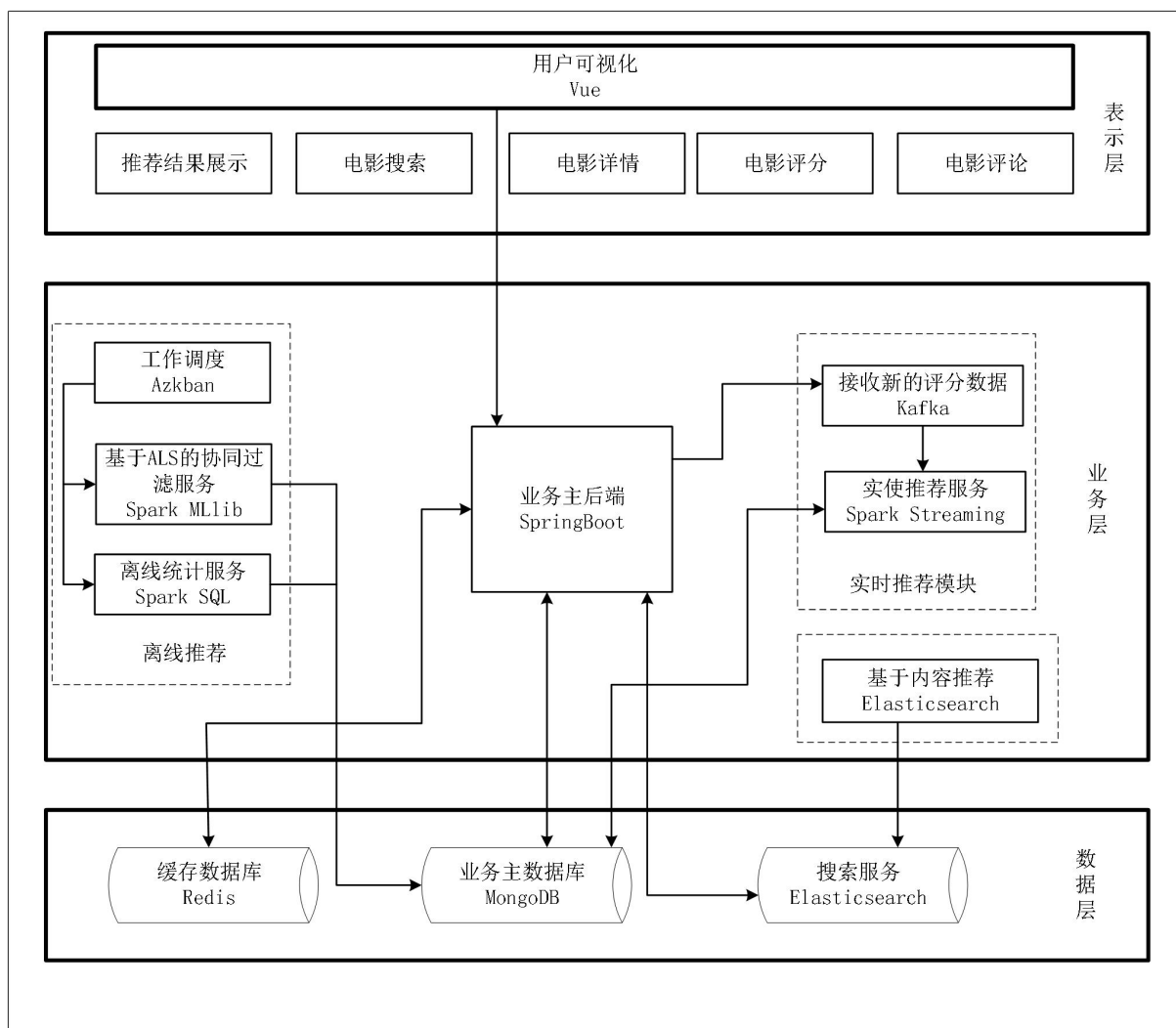


图 3.2 系统架构图

(1) 表示层：对应系统中的用户层，包括推荐结果展示、电影搜索、电影详情、电影评分、电影评论等模块。这些模块负责与用户进行交互，展示数据和接收用户输入，提供了用户与系统交互的界面。

(2) 业务层：对应系统中的业务层，包括实时推荐模块、离线推荐模块、基于内容推荐模块、电影信息查询服务等。这些模块处理具体的业务逻辑，如推荐算法、数据统计、搜索等，是系统的核心部分。

① 离线推荐模块

- 1) 任务调度：使用 Azkaban 调度离线统计服务和离线推荐服务。
- 2) 离线统计服务：主要使用 Spark 中的 Spark SQL 功能去业务数据库中加载评分表和电影表，然后计算出最近评分最多的电影（热点推荐）、结合所有历史数



据的历史评分最多电影，每个类别评分最多的电影。计算好之后再存放到业务数据库中。

- 3) 离线推荐服务：从业务数据库中加载数据，使用 Spark MLlib 中的 ALS 算法生成用户推荐矩阵和影片相似度矩阵，并将结果回写到业务数据库中。

② 实时推荐模块

- 1) 评分数据的接收：业务后端充当消息队列生产者，用户对电影评价之后将用户评分该行为数据发送到消息队列 Kafka，

- 2) 实时推荐计算：大数据处理模块充当消息队列消费者利用 Spark Streaming 监听 Kafka 队列，融合 Redis 中的用户最近评分数据，使用实时推荐算法计算新的推荐结果，并与 MongoDB 数据库进行交互。

- ## ③ 基于内容推荐模块：使用 Elasticsearch 中自带的 TF-IDF 算法根据用户感兴趣标签做基于内容的推荐。

- ## ④ 业务后端：使用 SpringBoot 和数据层进行交互完成平台的数据流动调配。

（3）数据层：对应系统中的数据层，包括缓存数据库（Redis）、业务主数据库（MongoDB）、搜索服务（Elasticsearch）等。这些组件负责数据的存储和管理，为业务逻辑层提供数据支持。

3.3 系统模块设计

3.3.1 用户基本业务模块设计

本文所说的用户基本业务模块指的是本系统除开推荐相关的内容外的业务，即除开核心业务还需要保证用户能够正常使用系统提高用户体验。对应活动图如图 3.3 所示：

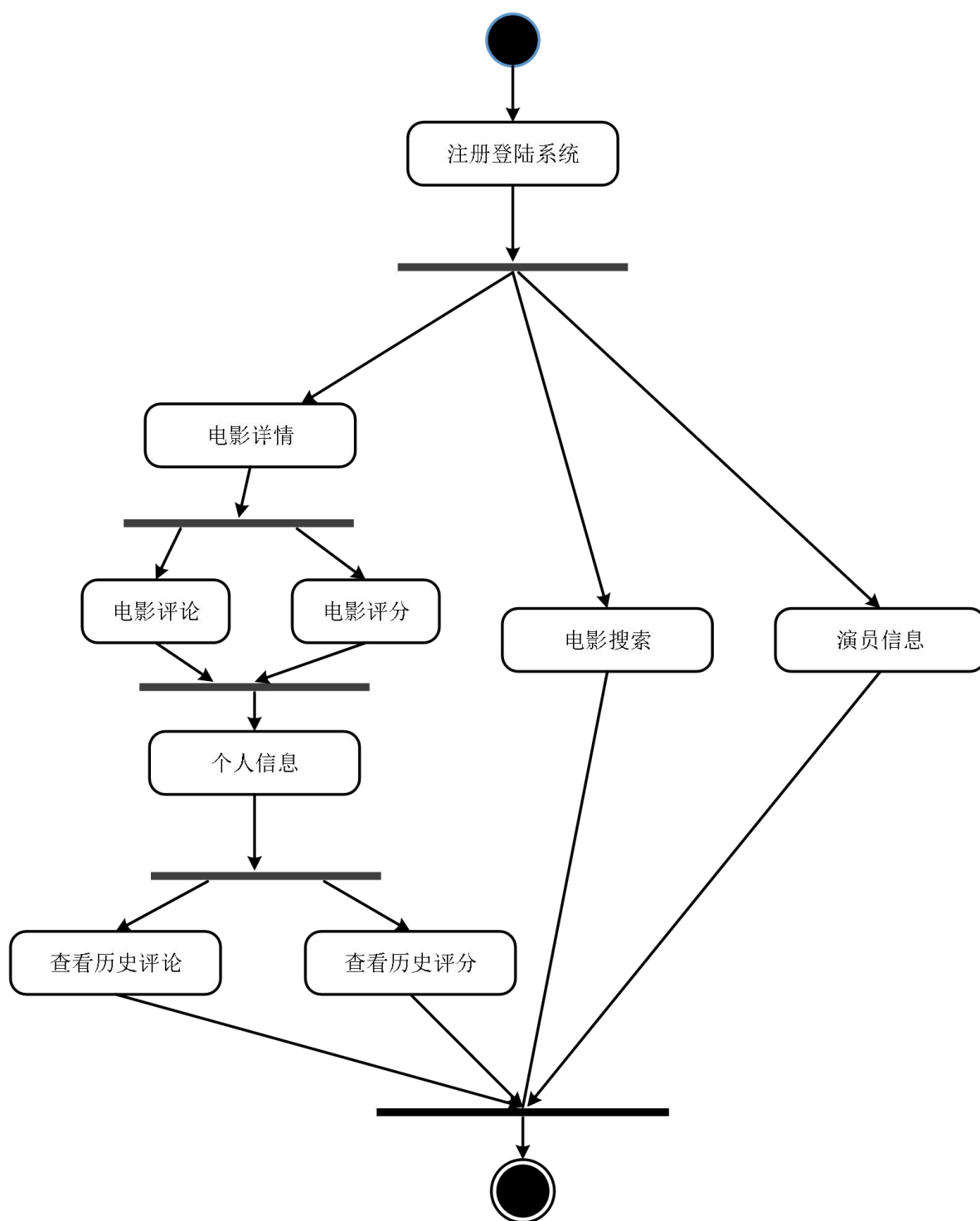


图 3.3 用户基本业务模块活动图

基本业务讲解如下：

（1）用户注册：游客访问系统首页后可以点击注册按钮跳转到注册页面，游客填写自己的用户名及其密码后前端会将对应的注册信息发送到后端，后端首先进行用户名



校验查看该用户是否已经注册过，如果没有那么后端需要加密用户的密码后才能保存到业务数据库 MongoDB 中来保证用户的信息的安全。

（2）用户登录：主要作为系统提供个性化服务的起点。作用为检验该用户是否为系统的合法用户。同样用户进入首页后点击登录按钮跳转到登录界面然后输入对应的用户信息包括用户名及其密码后点击登录前端会将对应的用户信息发送到后端，然后后端会使用注册时相同的加密方式对用户密码进行加密然后再去数据库进行查询。如果校验用户合法那么跳转首页，否则提示用户名或者密码不合法。

（3）基本业务：用户进行登录之后进入系统主页面，点击对应的电影可以查看电影详情，之后用户在浏览电影之后可以对电影进行打分评分范围在一到五分之间。然后针对于本电影用户在电影详情页面往下滑动可以看到关于该电影的评论相关信息，并且提供一个表单项给用户发表自己对于本电影的评论。之后用户可以在电影首页中点击“个人中心”然后找到“历史评论”以及“历史打分”两个导航栏就可以查看自己的历史行为数据。并且支持用户修改自己的历史行为数据，比如删除历史评论操作。同时用户也可以在主界面进行电影的搜索查找来定向查询电影以及点击“演员信息”可以查看系统中演员的相关详情。

（4）冷启动问题：因为个性化推荐系统主要是通过用户的行为数据来进行推荐的，所以当有新用户或者新电影上架时这时新用户和新电影此时是没有对应的相关的历史行为数据的，所以本系统采用当系统检测到用户为新用户的时候，系统会弹出一个选择框供用户选择自己喜欢电影的类别然后保存到数据库中。同时对于新电影来说同样在电影进行上架之前管理员需要给电影设置电影的对应类别。然后在基于用户推荐的时候就通过 TF-IDF 来将用户兴趣标签和电影的类别进行基于类容的推荐。同理在基于电影的推荐是将新电影的电影类别和系统的电影类别进行 TF-IDF 的基于内容的电影推荐。

3.3.2 离线推荐模块设计

离线推荐：主要针对于大数据量的计算，因为基于 ALS 的协同过滤核心是计算得到用户相似电影相似矩阵和基于电影的电影相似矩阵，计算较为复杂。针对于基于 ALS 的协同过滤算法在 1.4.3 章节中已经详细讲解这里不做赘述。如果每次用户调用推荐接口都要去运行一遍这个流程，那么就会造成系统资源过载，并且响应时间较长。所以本文针对于这种情况设计了离线推荐功能，既针对于大规模历史数据进行批量处理和分析

我们不需要实时的计算得到对应的矩阵或者统计结果集，只需要周期性的更新结果集就可以，比如以一天或者小时为单位去执行离线操作然后将最后的结果集更新到数据库中，因为对于整体上的历史数据，周期时间内收到的新数据对于整体来说影响不会很大，那么针对于推荐的时候就可以直接从数据库中获取。离线推荐活动图如图 3.4 所示：

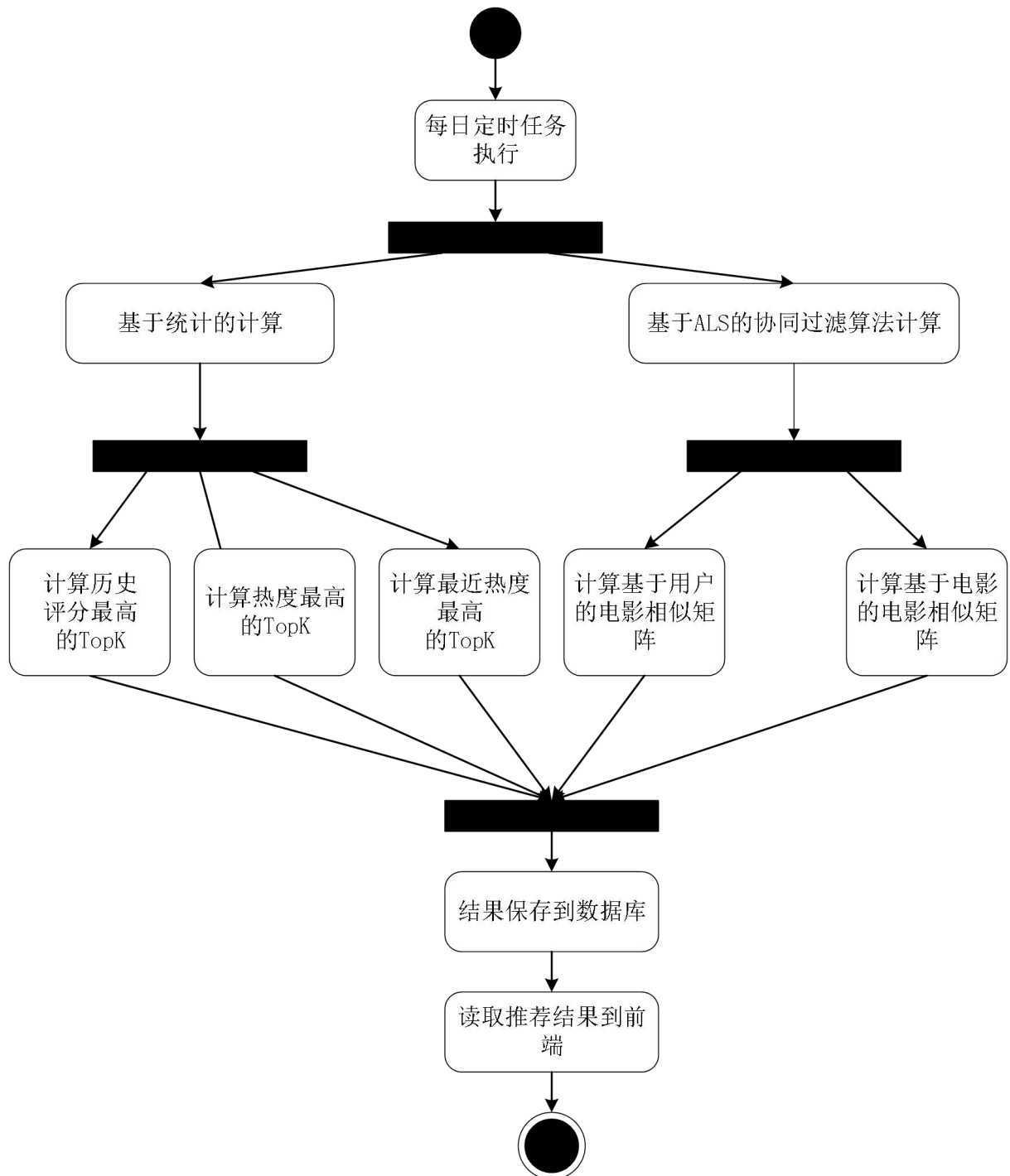


图 3.4 离线推荐模块活动图

其中定时任务调度采用的是 Azkaban 任务调度器完成，设置任务调度器每天定时在系统业务不繁忙的时候运行该离线任务。任务执行时，首先会先去 MongoDB 中加载里是评分表，然后进行相应的算法计算，包括基于统计的计算得到历史评分最高电影数据信息、热度最高以及最近热度最高等推荐信息和基于 ALS 的协同过滤的离线计算。

3.3.3 实时推荐模块

实时推荐系统的设计需同时满足两大核心需求：用户评分后推荐结果须即时响应更新，且需精准捕捉用户近期兴趣偏好。传统离线推荐算法（如 ALS）在实时场景下存在根本性局限——其矩阵分解计算过程具有高时间复杂度，难以满足毫秒级响应要求；更关键的是，ALS 算法仅通过增量更新全局评分矩阵实现推荐结果迭代，单次评分变动对整体相似度分布的扰动效应微弱，导致用户感知层面推荐列表缺乏显著变化，这种“更新延迟假象”将直接损害交互体验。

为突破上述瓶颈，实时推荐引擎需构建轻量级计算框架。用户兴趣演化规律表明，用户的兴趣随着时间的推移而变化，距离当前时间越近的评分越能描述用户当前的兴趣爱好^[15]。基于此，引入时间敏感的动态调节机制：通过设计推荐优先级调节因子，将时间衰减效应与正向反馈激励进行耦合建模。其中，增强因子通过对数函数压缩高相关电影的评分频次，既保留偏好强度特征，又避免热门电影过度主导推荐；衰减因子则采用反向对数变换，使低评分关联电影的负面影响呈边际递减效应，防止个别差评导致推荐结果剧烈波动。取对数之后不会改变数据的性质和相关关系，但压缩了变量的尺度，数据更加平稳^[16]。

当用户 u 完成对电影 p 的评分操作时，系统将启动三阶段更新流程：首先基于电影相似矩阵（离线推荐中基于 ALS 协同过滤算法生成）筛选与 p 最邻近的 K 部候选影片；随后结合用户近期行为序列，通过加权评分模型计算各候选电影的实时推荐优先级权重；最终将新生成的优先级向量与历史推荐列表进行融合运算，采用保留策略确保优质推荐不被完全覆盖，同时通过优先级差值阈值控制实现显著性更新。以此来保证推荐结果的即时可见性变化，同时通过动态因子调节维持了推荐系统的稳定性与解释性。

对应的设计为：首先，获取用户最近的 K 个评分，记为 RK ；然后再获取和当前评分电影 p 相似的 K 个候选电影，记为 S ；其中 q 为候选电影集合中的任意电影。计算出候选电影的推荐优先级 E_{uq} ，计算公式如公式（3.1）所示：



$$E_{uq} = \frac{\sum_{r \in RK} sim(q,r) \times R_r}{sim_sum} + \lg \max\{incount, 1\} - \lg \max\{recount, 1\} \quad (3.1)$$

对应公式的含义如下表所示：

表 3.1 推荐优先级公式参数表

公式参数	含义	备注
R_r	用户u对电影r的评分	无
$sim(q,r)$	电影q,r的余弦相似度	无
sim_sum	记录符合相似条件的候选电影q出现的次数	候选电影q在最近评分电影相似矩阵中出现且q和评分电影相似度大于设定的阈值 0.6
$incount$	RK中与q相似且RK评分高的个数	评分高于 3 分
$recount$	RK中与q相似且RK评分低的个数	评分低于 3 分
$\lg \max\{incount, 1\}$	计算增强因子的影响	强化用户近期行为影响
$\lg \max\{recount, 1\}$	计算增衰弱因子的影响	强化用户近期行为影响

公式的意义如下：

该公式本质上是用户行为特征与电影属性特征的融合计算模型。基础项采用相似度加权评分平均法，通过遍历用户 u 近期 K 个评分记录构成的集合 RK ，提取与候选电影 q 相似度超过阈值 0.6 的电影集合。对于每部相关电影 r ，将其与 q 的相似度系数 $sim(q,r)$ 作为权重，对用户原始评分 R_r 进行加权求和，最终通过相似电影数量 sim_sum 进行归一化处理。这种设计使得推荐强度与用户历史偏好及电影间关联性形成非线性映射关系。

公式后两项构建了动态调节机制：增强因子采用对数函数对高相关正向反馈进行压缩激励，当候选电影 q 与用户近期高分（ ≥ 3 分）电影的相似数量 $incount$ 增加时，推荐优先级呈对数级增长，既强化了用户当前兴趣倾向，又避免热门电影过度主导推荐结果；削弱因子则通过反向对数变换对低相关负向反馈进行衰减处理，当 q 与用户近期低



评分（<3 分）电影的相似数量 recount 增加时，推荐优先级呈对数级降低，这种边际递减效应可防止个别偶然差评导致推荐结果剧烈波动。在计算完每个候选电影 q 的 E_{uq} 计算后，会生成类似于一个元素个数为 K 个元素形式为 key-value 结构即<电影 q 的 ID, q 的推荐评分>的新的电影推荐列表 NewRecommendList 。同时在本次实时计算推荐之前该用户可能已经有了一张老的实时推荐表（新用户没有） OldRecommendList 其元素个数以及元素形式和 NewRecommendList 一样同样是 key-value 结构即<电影 ID, 电影推荐评分>的列表元素个数同样为 K 个。

最后最新的实时推荐会结合新旧两个实时推荐列表进行合并并且将合并后的集合中的元素按照推荐评分进行排序最终选取评分最高的 K 个然后收集到 RecommendList 集合中作为用户 u 的实时推荐列表即如公式（3.2）所示：

$$\text{RecommendList} = \text{topK}(i \in \text{NewRecommendList} \cup \text{OldRecommendList}, \text{cmp} = E_{ui}) \quad (3.2)$$

3.3.4 基于内容的推荐设计

针对推荐系统面临的冷启动困境，本项目构建了基于内容特征的推荐模块，作为协同过滤与实时推荐机制的有效补充。传统基于 ALS 的协同过滤算法在用户-电影评分矩阵稀疏时，会因缺失足够交互数据导致推荐结果随机化；而引入动态调节因子的实时推荐系统，在新用户首次访问时同样面临行为序列缺失的启动瓶颈。为此，系统设计基于 TF-IDF 算法的内容推荐引擎，专门应对冷启动场景下的推荐服务断层问题。

该模块的核心机制在于电影内容特征的向量化建模与用户偏好的显式采集。当新用户完成注册流程时，系统前端将自动触发偏好采集弹窗，通过结构化表单引导用户选择感兴趣的电影类型。后端服务接收用户提交的偏好向量后，立即启动 TF-IDF 特征加权计算：随后计算各特征项在用户偏好与电影类别中的逆文档频率，生成特征权重，最后系统选取 TopK 个电影进行推荐；基于内容的推荐的时序图如图 3.5 所示：

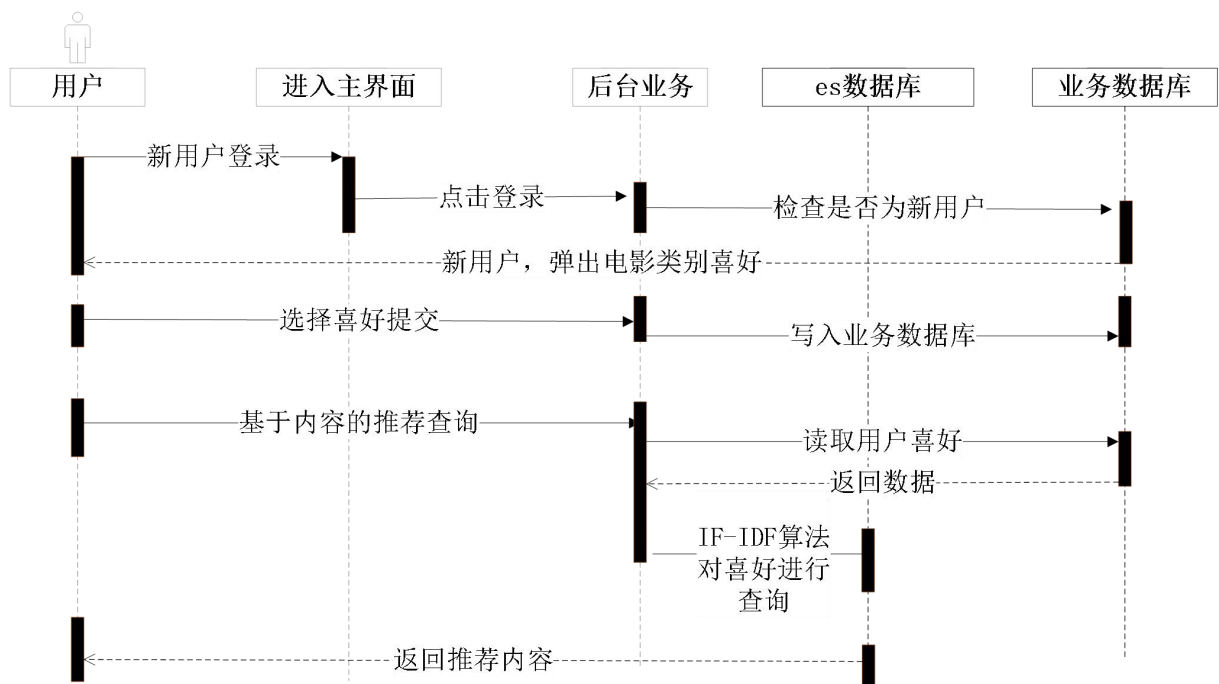


图 3.5 基于内容推荐的时序图

3.3.5 管理员总模块

管理员模块设计中主要是为了更好的进行系统管理，主要是对用户、电影以及用户评论进行管理，且本系统提供了统计分析即系统推荐转化率可视化统计，方便查看本推荐系统的作用。对应的管理员操作活动信息如图 3.6 所示：

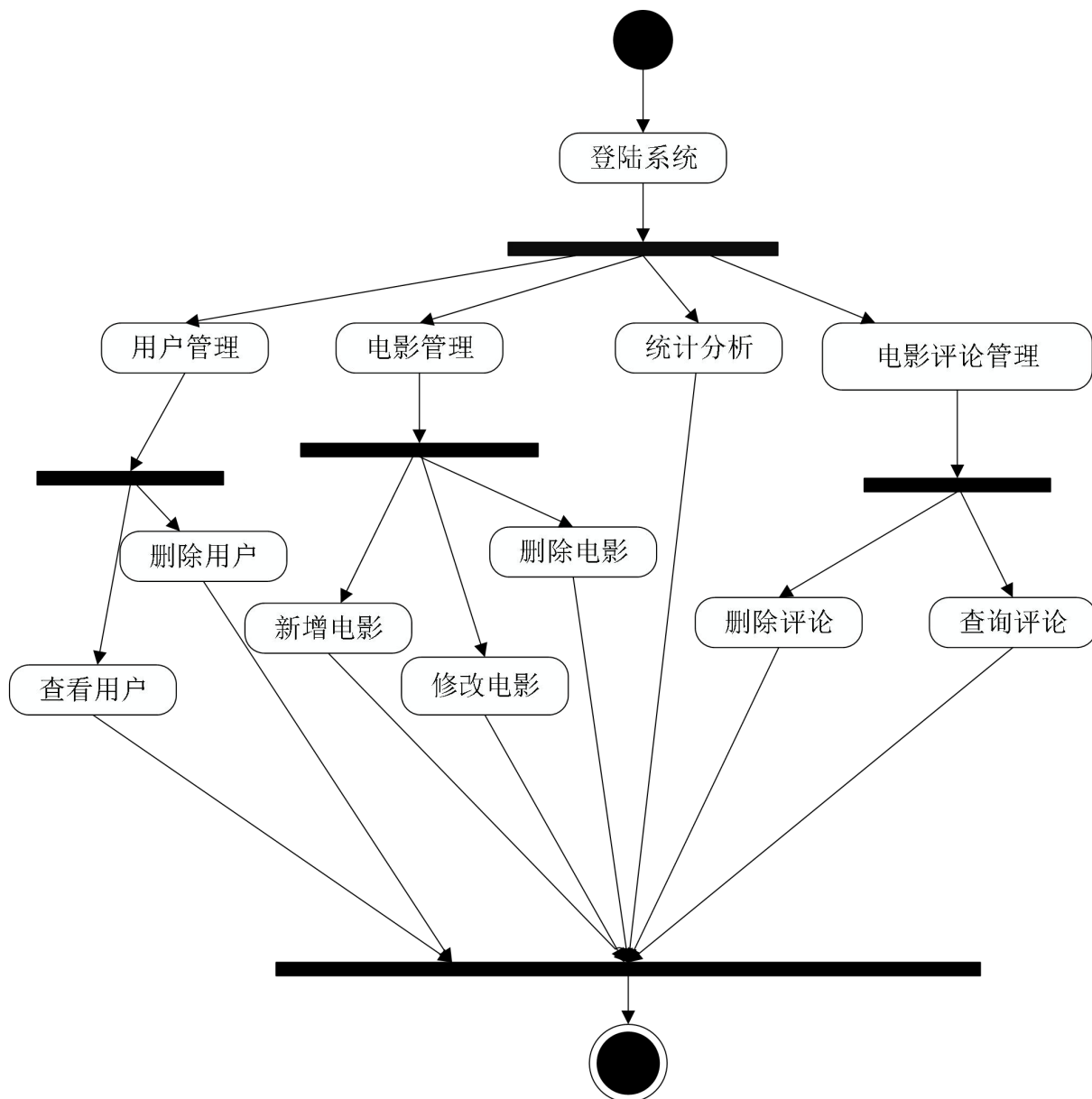


图 3.6 管理员总模块活动图

3.4 数据库设计

3.4.1 数据库的逻辑设计

本系统采用 MongoDB 作为后台主数据库，主要用于存储和管理数据信息。考虑到平台功能和需求之间的耦合程度，本系统数据库总共设计了十三张表。其中包括五张基本表和八张推荐表。五张基本表包括：用户表、电影表、评分表、评论表、演员表。其余为推荐表。对应的关系以及表中的属性如图 3.7 所示：

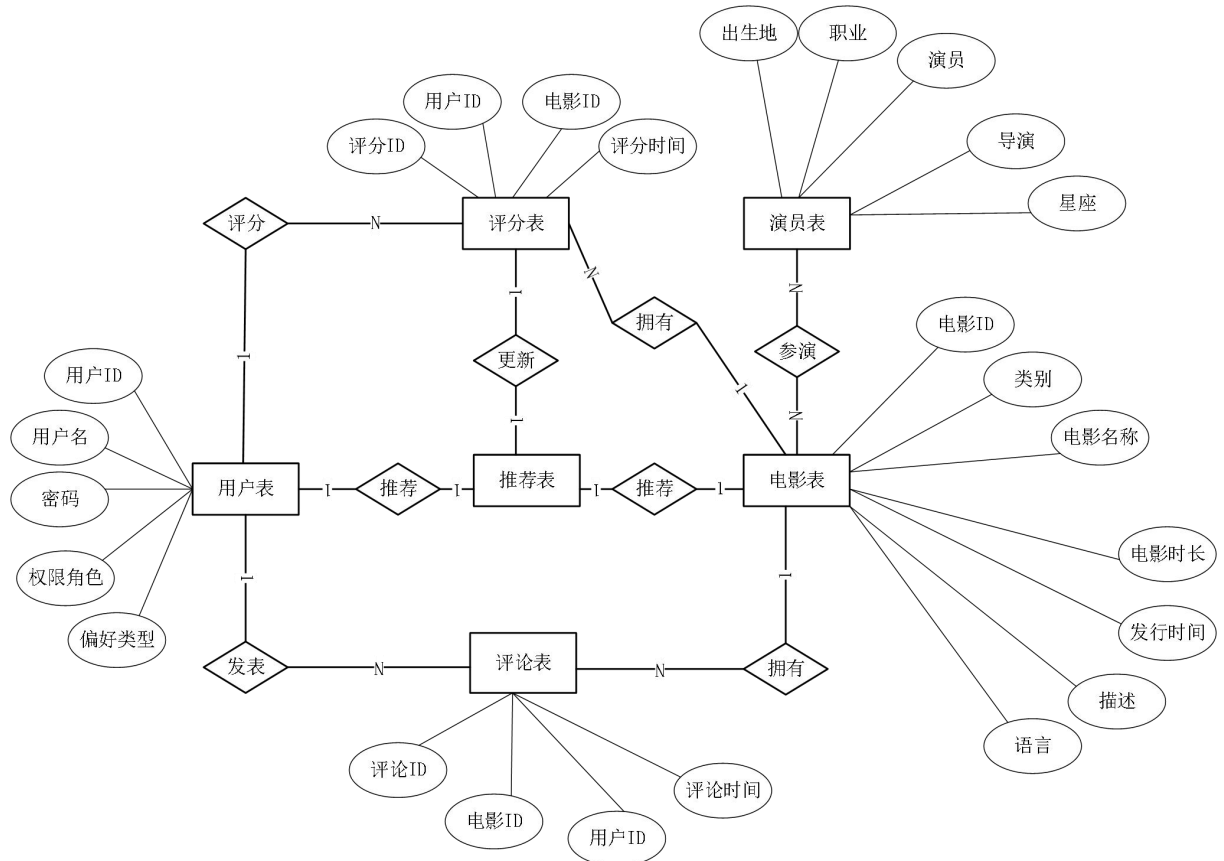


图 3.7 系统数据库 ER 图

3.4.2 数据库的物理实现

基本表实现如下：

Users 用户表用户保存用户的基本信息如表 3.2 所示：

表 3. 2 用户表

字段名	字段类型	字段描述
userId	Int	用户的ID
userName	String	用户名
password	String	密码（使用MD5 加密）
userRole	String	用户角色（user/admin）
prefGrenres	String	用户偏好
first	Boolean	是否为新用户
timestamp	Long	评分的时间

评分表用户保存用户对电影的评分，是用户-电影表多对多的关系表如表 3.3 所示：



表 3. 3 用户评分表

字段名	字段类型	字段描述
Id	Int	评分ID
userId	Int	用户的ID
movieId	Int	电影的ID
score	Double	用户对电影的评分
timestamp	Long	评分时间戳

Comments 评论表用于保存用户对电影的评论，是用户-电影表多对多的关系表。表结构如表 3.4 所示：

表 3. 4 评论表

字段名	字段类型	字段描述
Id	Int	评论ID
movieId	Int	电影的ID
userId	Int	用户ID
comment	String	评论
timestamp	Long	评论时间

movies 用于保存电影的基本评分如表 3.5 所示：

表 3. 5 电影信息表

字段名	字段类型	字段描述
movieId	Int	电影的ID
name	String	电影的名称
storyline	String	电影的描述
mins	String	电影的时长
releaseDate	Date	电影的发行时间
year	Int	电影的发布年份
language	String	电影的语言
genres	String	电影类别



续表 3.5 电影信息表

字段名	字段类型	字段描述
IsUp	Short	是否上架（0 下架，1 上架）

Persons 演员表用于保存电影中演员、导演的的基本信息。如表 3.6 所示：

表 3.6 演员导演表

字段名	字段类型	字段描述
PersonId	Int	演员的ID
name	String	演员名
birthPlace	String	出生地
constellation	String	星座
otherName	String	别名
profession	String	职业
sex	String	性别

Person_movie 表用于保存 person 表和电影表多对多的关系。如表 3.7 所示：

表 3.7 演员电影关系表

字段名	字段类型	字段描述
Id	Int	主键id
personId	Int	演员Id
movieId	Int	电影Id

基于离线统计的类别最高评分表如表 3.8 所示：

表 3.8 类别最高评分表

字段名	字段类型	字段描述
Id	Int	主键id
genres	String	电影类别
Recs	Array[(movieId:Int,score:Double)]	电影Id

基于离线统计的平均评分最高表如表 3.9 所示：



表 3.9 平均评分最高表

字段名	字段类型	字段描述
Id	Int	主键Id
movieId	Int	电影Id
avg	Double	电影平均评分

基于离线统计的最近评分最多表如表 3.10 所示：

表 3.10 最近评分最多表

字段名	字段类型	字段描述
Id	Int	主键Id
movieId	Int	电影Id
count	Int	评分总数
yearmonth	String	评分时段

基于离线统计的历史评分最高表如表 3.11 所示：

表 3.11 历史评分最高表

字段名	字段类型	字段描述
Id	Int	主键Id
movieId	Int	电影Id
count	Int	评分总数

基于用户的电影相似矩阵表如表 3.12 所示：

表 3.12 基于用户的电影相似矩阵表

字段名	字段类型	字段描述
Id	Int	主键Id
userId	Int	用户Id
Recs	Array[(movieId: Int, score: Double)]	相似电影集合

基于电影的电影相似矩阵表如表 3.13 所示：



表 3. 13 基于电影的电影相似矩阵表

字段名	字段类型	字段描述
Id	Int	主键Id
movieId	Int	电影Id
Recs	Array[(movieId:Int,score:Double)]	相似电影集合

实时推荐表如表 3.14 所示：

表 3. 14 实时推荐表

字段名	字段类型	字段描述
Id	Int	主键Id
userId	Int	用户Id
recs	Array[(movieId:Int,score:Double)]	实时推荐电影



第四章 系统详细设计

系统开发遵循原型化设计原则，采用分层架构实现前后端协同运作。整体技术栈构建于前后端分离的架构体系之上，将系统功能解构为前台用户交互层与后台管理服务层两大核心模块，通过数据库中间件实现全链路数据贯通。

在功能维度划分上，系统运作呈现三级处理流水线：前端界面层承担用户请求的接收与可视化呈现，通过响应式设计支持电影检索、评论交互、评分系统等核心功能；后台业务主模块作为中枢处理引擎，集成用户认证、权限管理等基础服务，同时对接大数据处理层；Spark 计算集群作为独立的数据处理层，专门承担推荐算法的并行计算任务，通过内存计算框架实现实时特征提取与模型推理。

架构设计层面采用 B/S 架构，管理员通过安全认证后进入后台管理系统，可执行用户数据维护、电影评论管理、电影管理等全维度管控操作；用户前端采用微服务架构设计，注册用户通过门户网站访问个性化服务。其交互通过 ORM 框架实现数据库操作的标准化封装，采用消息队列 Kafka 解耦前后端请求，最终由 Spark Streaming 实现实时计算与批量处理的混合调度。系统主要类图如图 4.1 所示：

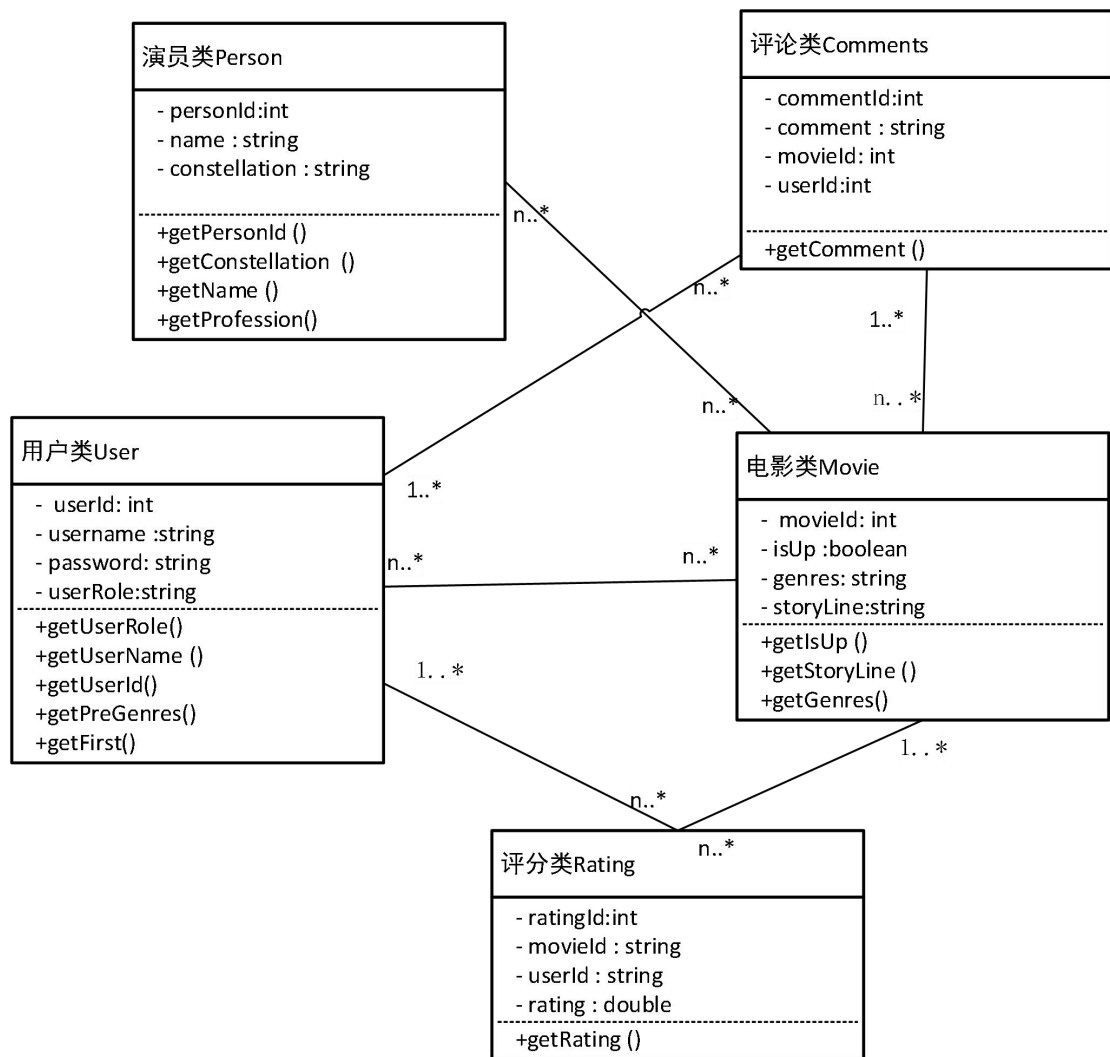


图 4.1 系统主要类图

4.1 用户基础业务模块设计与实现

(1) 用户注册设计

用户注册是系统构建个性化服务生态的核心前提，其价值直接体现在服务权限的差异化配置中。当用户选择以游客身份访问时，系统将自动启用基础服务模式，该模式仅开放通用性功能模块，包括标准化的电影检索引擎、热点推荐以及公开演员信息等基础服务。用户无法享用个性化推荐，以及与系统的交互活动比如对电影进行评分、评论等功能。注册流程如图 4.2 所示：

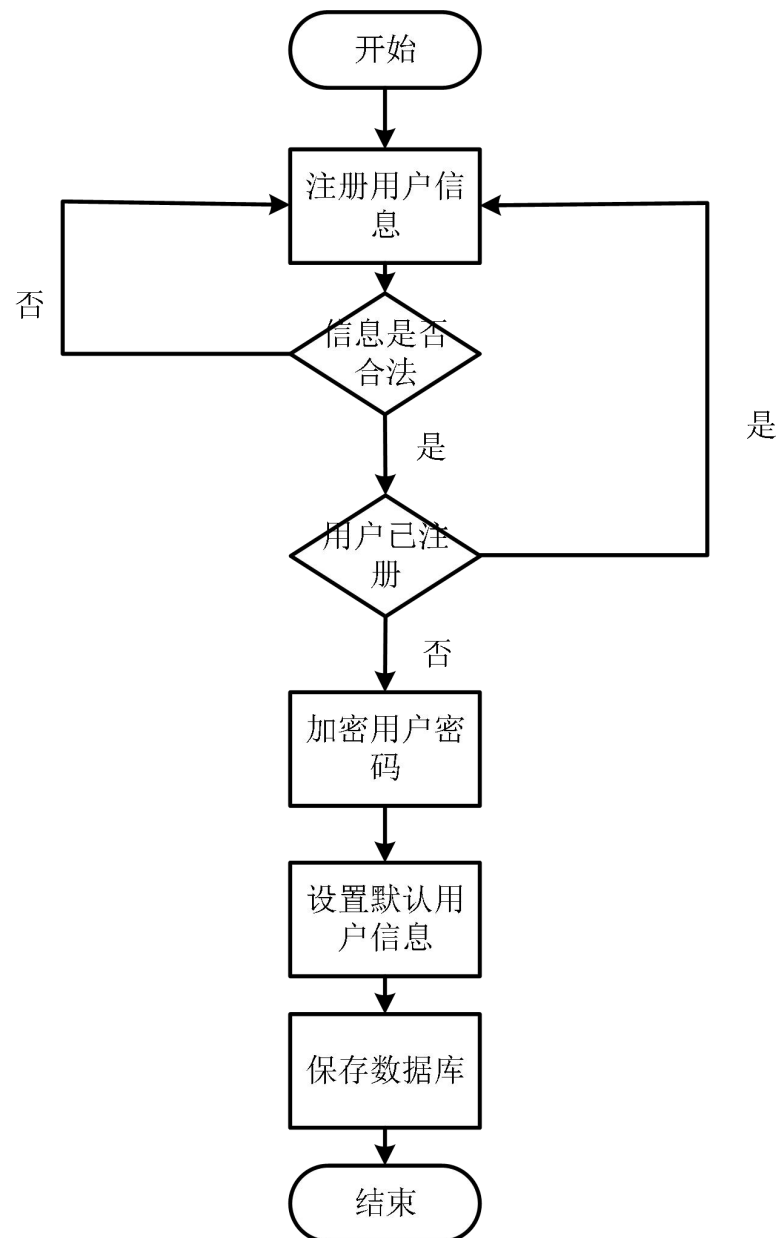


图 4.2 用户注册流程图

结束在前端实现注册界面，包括表单和提交按钮、用户填写用户名和密码后点击提交按钮前端通过 ajax 中的 post 方法向后端发送请求。在后端使用 @PostMapping 映射，然后通过 @RequestBody 来解析对象。首先后端会通过 userService.checkUserExist() 方法判断用户是否已经注册过了，如果注册过了那么通过 ResponseMessage.failedMessage("该用户名已被注册") 同一封装错误信息然后返回前端。如果是第一次注册的，那么后端会通过 MD5 的方式将用户密码进行加密，以及默认设置用户权限为 user，并且将用户 first 字段设置为 true 表示该用户是新用户，为后面弹出新用户填写兴趣表单做准备。最

终通过 `collection.insertOne(user)` 数据库交互方法将用户信息保存在数据库中

（2）用户登录设计

系统登陆流程图如图 4.3 所示：

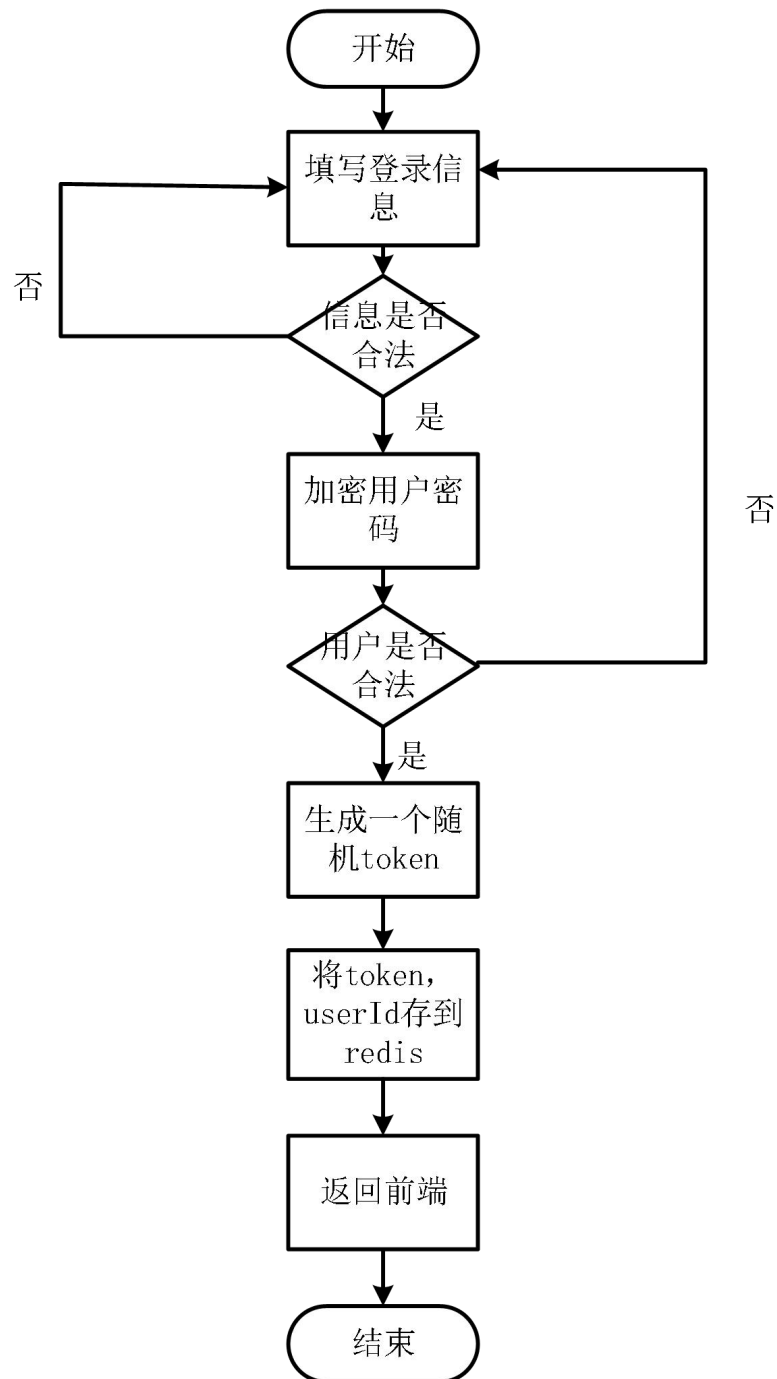


图 4.3 用户登录流程图

首先后端通过 `@Post Mapping` 映射方法以及 `@RequestBody` 方法接收到前端用登录界面的请求参数后。首先后台系统会将用户密码使用和注册时候相同的 MD5 算法进行



加密，之后通过 MongoDB client 交互工具和数据库交互，调用 `collection.find()` 进行条件查询，如果通过用户名、加密后的密码查询不能查出用户那么统一调用 `ResponseMessage.failedMessage()` 方法返回错误信息给前端，否之表示用户登录合法，那么会将产生一个随机 token 保存到 redis，其中 key 为 token，value 为 userId，用于完成用户的分布式登陆问题。最后将用户信息以及对应的 token 通过 `ResponseMessage.successMessage()` 返回前端。

（3） 电影搜索功能

用户通过填写想要搜索的电影名称、发行时间以及可以选择上要筛选的电影类别。后通过点击搜索，将对应的搜索条件传到后端。后端接收设计一个接收对象，其中由于有对时间的范围查询，后端设计 `rangeDate` 字段数据类型为数组，即 `Date[]`。通过 `@RequestBody` 获得查询条件，然后通过 `movieService.filterMovies()` 进行条件查询最后通过分装条件通过 `collection.find()` 进行 MongoDB 的条件查询进行筛选后将获得的结果返回给前端，前端获得数据之后进行分页展示。

（4） 查看演员相关信息

用户点击演员信息时，前端会将该演员对应的 `personId` 传送到后端，然后后端通过 `movieCollection.find()` 查询到对应的演员信息然后再返回给前端。之后再通过演员名称在电影集合中通过演员名称进行查询即通过 `personCollection.find()` 找到该演员演过、导演过的电影然后进行展示。具体时序图如图 4.4 所示：

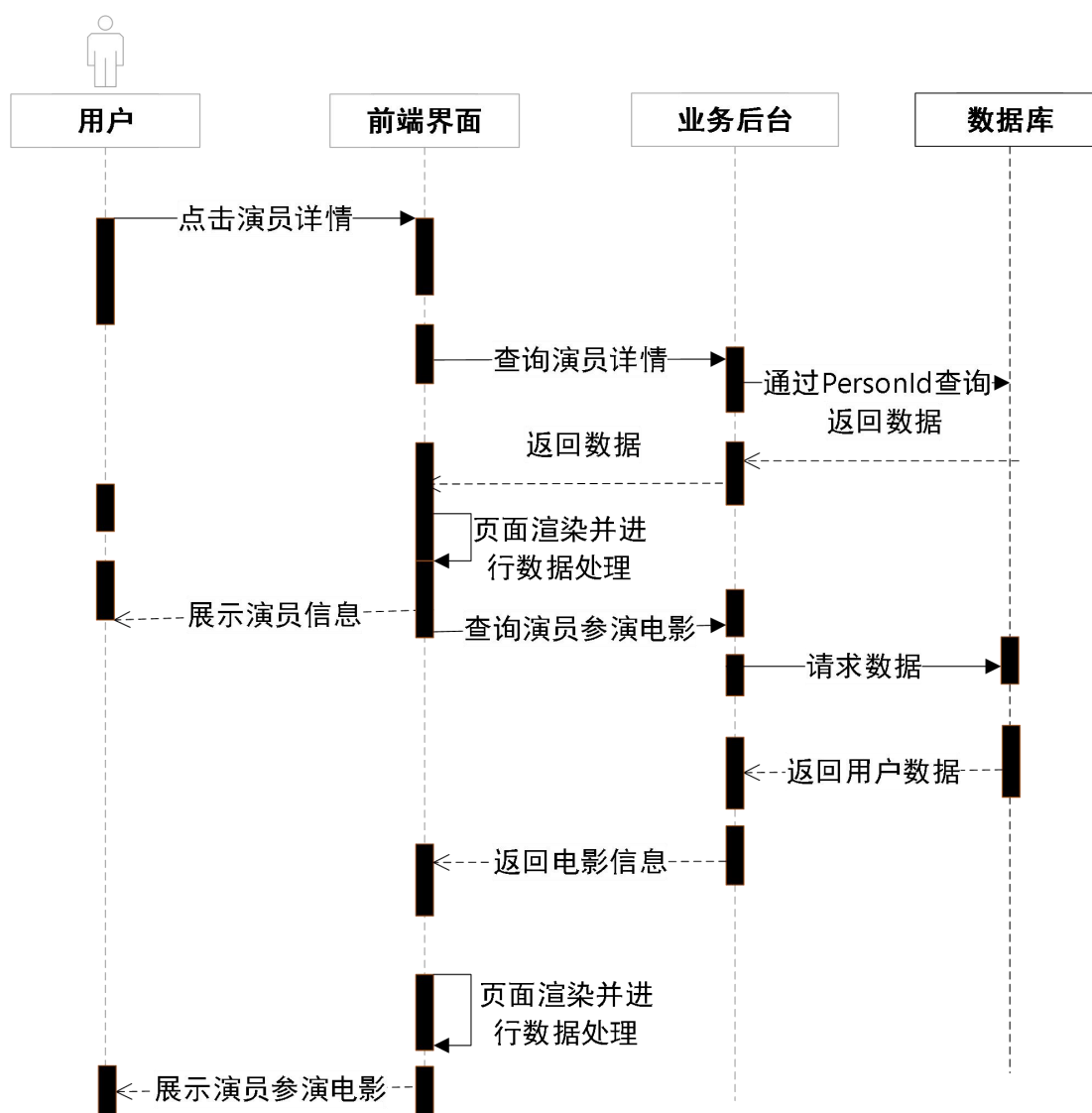


图 4.4 查看演员相关信息时序图

(5) 电影详情：前端通过传入用户点击的电影的 `movieId` 然后后端 `MovieController` 接收到参数之后然后调用 `movieService.fillterMovie()` 方法进行查询后将电影详情数据返回给前端，然后前端进行展示。

(6) 新增电影评论：前端提供一个文本框给用户填写电影评论，然后用户点击提交按钮后，前端会发送 Ajax 请求到后端接口。后端接收通过 `@RequestBody` 拿到用户信息和电影的 `movieId` 以及对应的评论信息后，`CommentController` 调用 `commentService` 中的 `addComment()` 方法进行插入。插入成功后返回信息给前端。

(7) 查询历史电影评论：用户点击“个人中心”中的“历史评论”信息后前端会将用户信息发送到后端，然后后端通过 `CommentController` 调用 `commentService` 中的



filterComent()方法按照用户 ID 进行过滤后将查找到的数据返回给前端。

（8）电影评分：用户在进入“电影详情界面”之后，前端会先发送将用户信息以及浏览的电影 ID 发送到后端，然后后端会先查询该用户是否已经评分过了，如果没有评分那么前端会提供 1-5 星提供给用户评分，如果之前已经打过分了那么前端则显示重新评分按钮。之后用户点击对应的评分后，前端会将用户信息、电影 ID 以及评分信息发送给后端，然后后端会先判断该用户对于该电影是否评分过。如果已有记录则更新用户对于该电影的评分，否则将用户评分信息插入到评分表中，同时将用户评分保存在 Redis 的 List 数据类型的集合中 Redis 中的 List 的 Key 为 value 格式为“movieId:score”，最后服务端充当生产者将评分数据按照并通过分隔符“||”记录用户 ID、电影 ID、评分以及时间戳信息发送到 Kafka 消息队列。

（9）冷启动问题解决：首先用户在登录系统之后后端会返回用户的相关信息给前端，然后前端就可以拿到用户信息中的 first 字段，通过 first 字段值是否为 true 来判断用户是否为新用户，如果是新用户那么前端自动弹出一个选择框给新用户选择自己所喜欢的电影类型，当用户点击提交之后前端会将包含用户信息和用户电影偏好选择集合发送给后端，然后后端节后到之后会将用户偏好分割后保存到数据库中，最后将数据库中该用户信息中的 first 字段改为 false。冷启动解决流程图如图 4.5 所示：

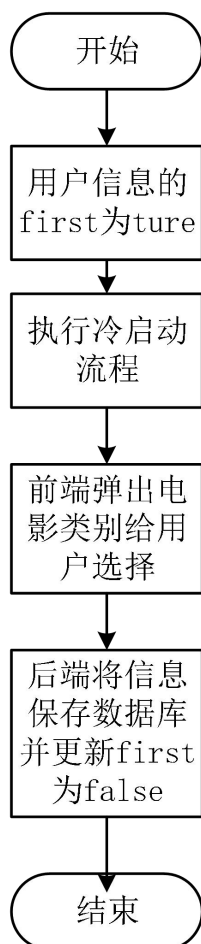


图 4.5 冷启动流程图

4.2 管理员模块设计与实现

（1）管理员鉴权设计：

对于后台管理来说，需要用户身份为管理员即 `userRole=admin` 时才能有权限进行管理，针对于管理权限设计本项目采用 Spring 中的 AOP 功能面向切面编程，即为需要管理员权限操作得方法做一个动态代理工作。在 Spring 中 AOP 的实现可以基于注解 `@annotation` 的形式完成。AOP 这里的通知顺序采用的是 `around` 环绕通知。主要步骤是：

用户访问后台管理相关接口后 AOP 会拦截到对应的请求后，首先会先会从 `HttpServletRequest` 请求中获取到用户请求相关信息拿到用户的 `token`，然后通过 `redis` 获取到该 `token` 对应的用户 `Id`，之后就可以获取到当前访问接口的用户，然后拿到用户信息中的权限即 `userRole` 字段，然后判断如果 `userRole=admin` 那么通过 AOP 中的 `joinPoint.proceed()` 为请求放行，然后请求就能进入到后端的业务接口部分。如果判断 `userRole=user` 那么会向前端抛出“无权限”的异常，对应的流程图如图 4.6 所示：

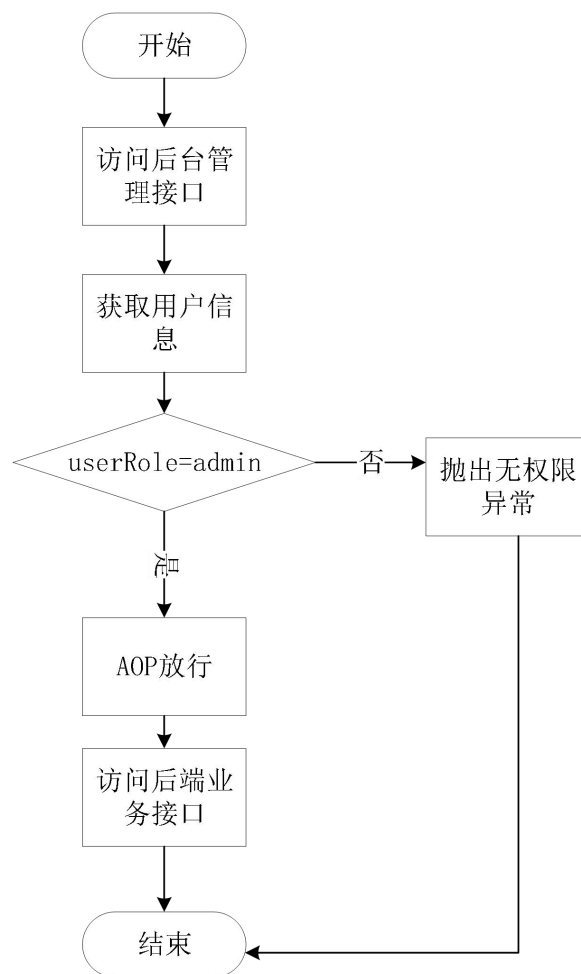


图 4.6 管理员鉴权设计流程图

（2）用户管理：在删除之前请求会先进行过基于注解式的 AOP 进行鉴权处理。管理员可以通过点击用户管理模块查询到对应的用户信息，然后通过批量选择、或者单个选择然后前端得到想要删除的 `userId` 集合，然后 Ajax 像后端发送请求。之后后端通过 `@RequestBody` 获取到 `userId` 集合，然后通过 `userService.deleteUsers(ids)` 进行删除。过程中会对这一批要删除的用户先进行用户角色判断，如果含有删除的角色为 `admin` 的会抛出异常说明权限不够，因为本系统设计的是管理员不能删除管理员。

（3）电影管理：

前置权限处理：在执行电影管理相关 API 时同样会先进行过基于注解式的 AOP 电影上架下架：

在执行电影上架或下架操作时，系统会首先通过基于注解的 AOP 进行鉴权处理，验证当前用户是否具备对应操作权限（如 `MOVIE_SHELF_ADMIN` 角色）；管理员在电



影详情页或列表页点击状态切换按钮后,前端通过 Ajax 发送包含电影 ID 及目标状态(上架/下架)的 PUT 请求至后端,后端通过 @PathVariable 接收参数后,调用 movieService.isUpMovie(id, status)执行状态变更,

电影新增:在执行电影新增操作时,系统会首先通过基于注解的 AOP 进行鉴权处理,验证当前用户是否具备电影新增权限;管理员在电影管理模块填写完整的电影信息后,前端通过 Ajax 将包含电影名称、分类、导演等数据的 JSON 请求发送至后端,后端使用 @RequestBody 收并解析数据,调用 movieService.addMovie(movieInfo)完成电影新增,过程中会自动校验必填字段及数据格式合法性。

电影批量删除:进行电影批量删除时,系统同样会先通过基于注解的 AOP 进行鉴权处理,确保操作者拥有电影删除权限;管理员在电影列表中选择待删除影片(支持批量勾选或单选),前端将收集的电影 ID 集合通过 Ajax 发送至后端,后端通过 @RequestParam 接收 ID 数组后,调用 movieService.deleteMovies(ids)执行删除操作,系统会在删除前自动校验电影状态及关联关系,确保操作合法性。

(4) 评论管理:在执行评论管理操作时,系统会首先通过基于注解的 AOP 进行鉴权处理,验证当前用户是否具备评论管理权限(如 COMMENT_ADMIN 角色);管理员在评论管理模块中可查看所有评论列表,支持通过关键词搜索、筛选状态(如待审核、已通过、已驳回)快速定位目标评论,可通过批量勾选或单选方式选择需处理的评论,前端将收集的评论 ID 集合通过 Ajax 发送至后端,后端通过 @RequestBody 接收 ID 数组后,调用 commentService.processComments(ids, action)执行批量操作(如删除、审核通过、标记违规),过程中会自动校验评论状态合法性(如已删除评论不可重复操作)及内容合规性,确保操作符合平台规则。

(5) 推荐转化分析可视化:

该功能主要为了提供给管理本系统的推荐电影功能效果展示,核心思路就是去记录每个月给用户推荐电影的有效转化率如公式(4.1)所示:

$$R = \frac{n}{N} \times 100\% \quad (4.1)$$

其中 n 表示用户每个月份点击推荐相关电影计数, N 表示当月给用户推荐的所有电影个数。 R 表示系统当月的推荐电影有效转化率。所以该可视化分析功能核心为计数。



本系统采用的方案是使用 Spring 中的 AOP 切面技术结合 Redis 来实现，选用 AOP 的原因为让统计分析不侵入原来业务使得功能解耦增强系统扩展性和健壮性。

使用 Redis 完成统计落地方案如下：

本系统使用 Redis 数据结构以及结构如图 4.7 所示，即首先使用 Redis 的 Set 结构接收单个用户每个月所受到的所有推荐电影的 Id 以及用户实际点击推荐电影的 Id，这两个 Set 集合伴随着用户点击电影详情的时候会进行动态更新。然后系统核心是需要对总的推荐转化率做统计而非单个，所以增加两个数据结构 Redis 里面的 Hash 结构来存放总的统计，里面的每一个 Field 为用户 Id，每个 Field 对应的 value 为当前 userId 下用户当月的总的点击推荐电影的总次数和系统推荐给用户的总的推荐电影数，最后统计两个 hash 中的次数中和相比即得到对应的转化率。

存放用户每月点击推荐电影的 movieId 和所收到推荐的所有 movieId 对应 Set 结构的 key 设计如下：

总推荐 Key: movie:analysis:total:all:currentTime:year:month:userId

有效推荐 Key: movie:analysis:real:all:currentTime:year:month:userId

系统总记录每月推荐转化的 Hash 结构的 key 设计如下：

总推荐 Key:movie:analysis:len:all:currentTime:year:month

有效推荐 Key:movie:analysis:len:real:currentTime:year:month

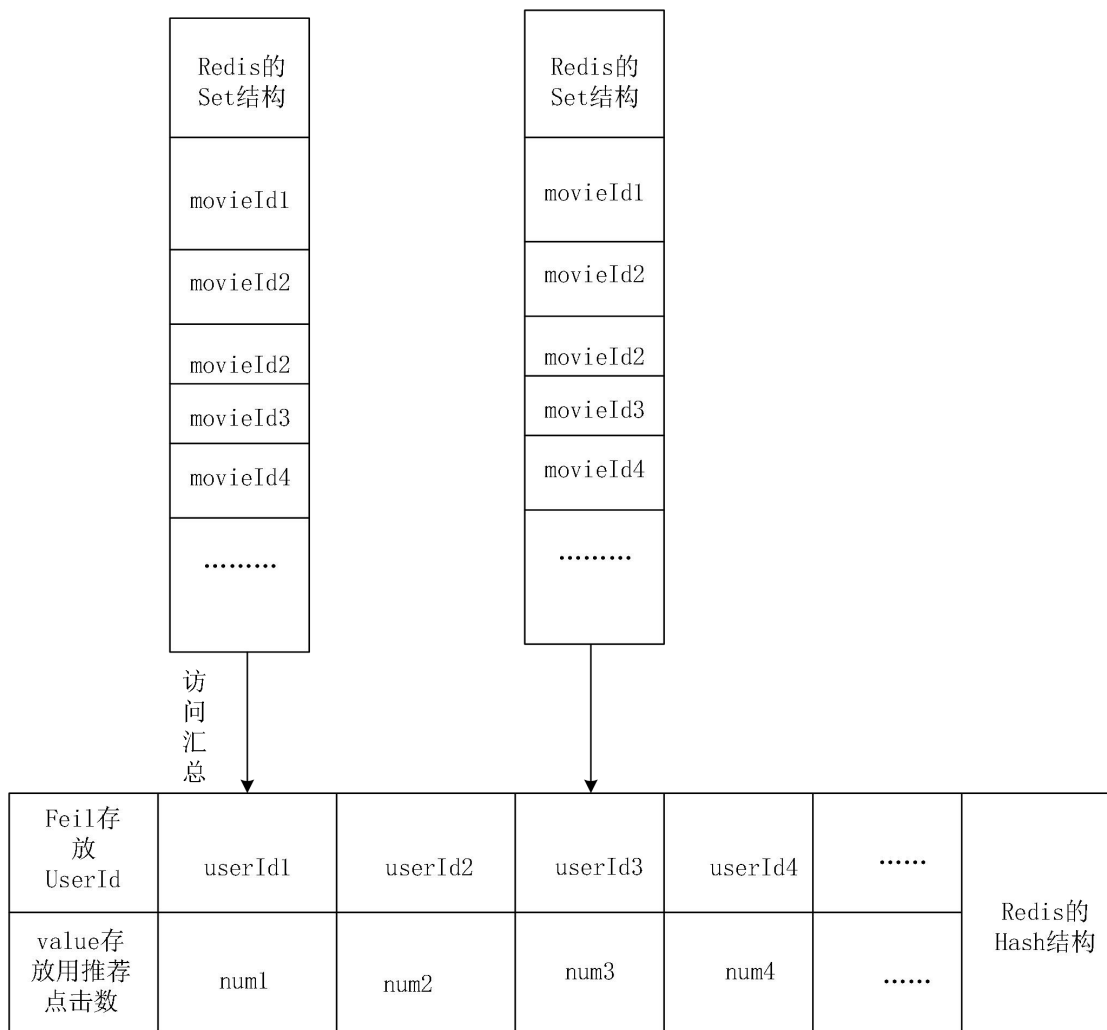


图 4.7 推荐转化率存储结构设计图

实际业务 AOP 设计如下：首先用户访问电影详情接口时 AOP 会对请求进行拦截并且会从用户请求中拿到用户请求的相关信息比如 userId 以及 movieId，然后 AOP 会去 MongoDB 中拿到给用户的实使推荐数据库 StremaRecs 中该用户推荐的所有 movieId 并经这些 movieId 存入 redis 的 Set 集合中，同时判断用户当前访问的电影是否为推荐的电影，如果是那么再将该 movieId 放入 Redis 的有效推荐集合列表中。处理完计数之后就将请求放行。最后当管理员进行推荐转化可视化分析的时候会访问后端的/move/analysis 路径下的 getAnalysis 接口得到系统每个月份的有效推荐转化率然后前端使用 Echarts 代码进行可视化展示。具体推荐转化率分析时序图如图 4.8 所示：

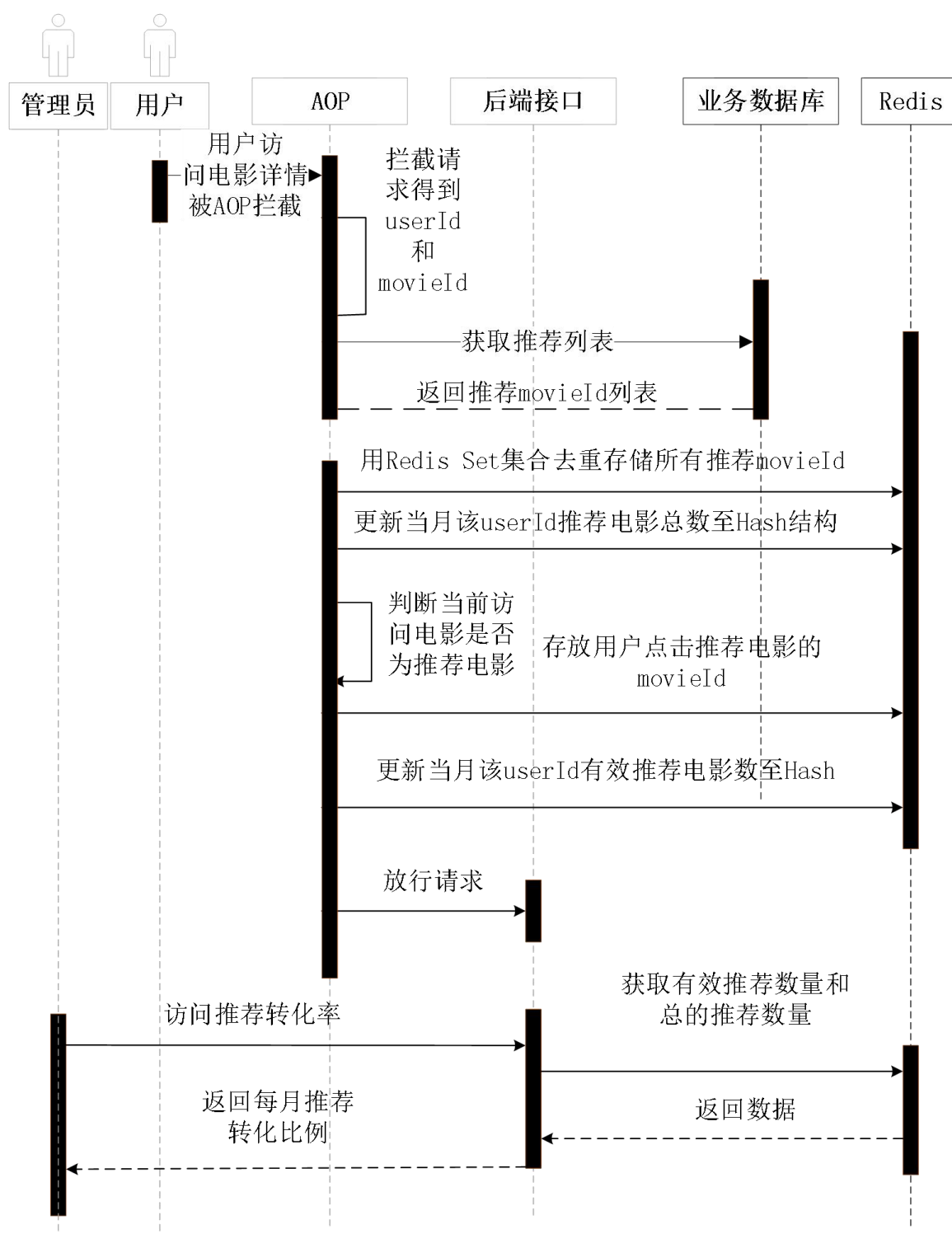


图 4.8 推荐转化率业务时序图



4.3 离线推荐模块设计与实现

4.3.1 基于离线统计的推荐设计与实现

首先通过 SparkConf 配置本地化集群参数并创建 SparkSession 实例，初始化分布式计算引擎后，采用 spark.read.option().format().load()方法从 MongoDB 并行加载评分数据和电影元数据，分别映射为 Rating 和 Movie 样例类的 DataFrame 对象，同时注册 ratings 临时视图以支持结构化查询。在核心统计模块中，采用四路并行的计算策略：历史热门统计通过 spark.sql("SELECT ... GROUP BY")执行聚合函数 COUNT()统计总评分次数，直接反映电影的长期受欢迎程度；近期热门统计则借助 SimpleDateFormat 和 spark.udf.register("changeDate")注册用户定义函数（UDF），将时间戳转换为年月格式后，通过 GROUP BY 按年月和电影 ID 进行分组实现动态时间窗口的评分热度计算；优质电影统计利用 AVG()聚合函数计算平均分，通过 groupBy().agg()生成质量标杆数据。在类别推荐模块中，通过 sparkContext.makeRDD().cartesian()生成全量类型与电影数据的笛卡尔积，结合 getAs[String]().contains()进行字符串匹配过滤，随后使用 groupByKey().map()对组内电影按平均分降序排列，最终通过 take(10)截取 TopN 结果并封装为 GenresRecommendation 结构体。所有统计结果均通过 storeDFInMongoDB()方法，采用 overwrite 模式调用 df.write.option().mode().format().save()持久化到 MongoDB 的独立集合中，整个流程以 Spark 内存计算引擎驱动，通过 RDD 转换算子和 DataFrame 结构化 API 实现高效并行统计。离线统计活动图如图 4.9 所示：

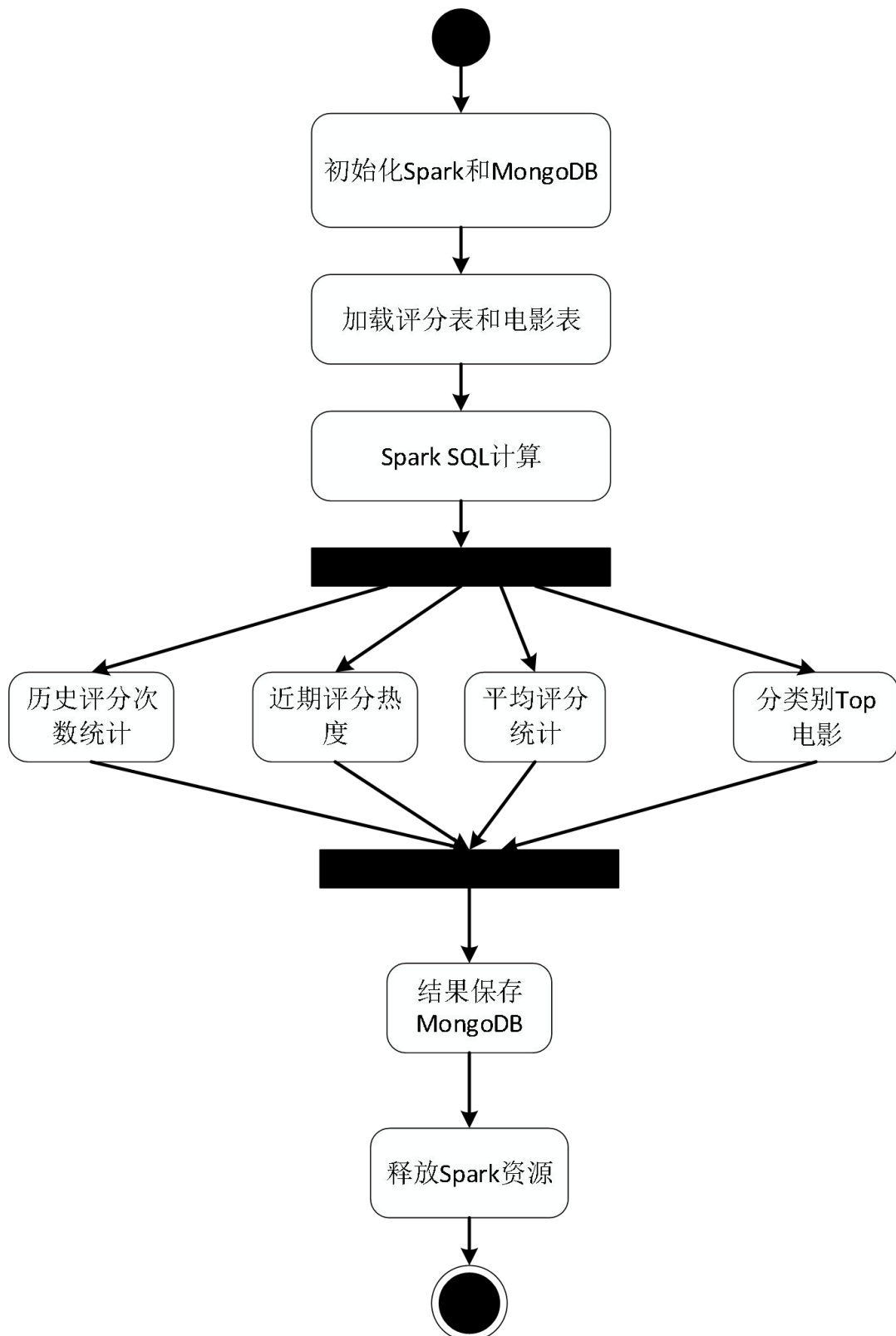


图 4.9 离线统计活动图

4.3.2 基于 ALS 协同过滤推荐设计与实现

对于基于 ALS 的协同过滤算法的原理以及对于预测误差的计算方式 RMSE 在章节



1.4.3 已经讲述，这里不做赘述。本小节主要讲述使用 Spark MLlib 库来具体实现。实现主要分为两步：第一为模型训练，即通过调试 ALS 模型，使得预测评分表和真实评分表的误差尽量小，从而取得误差较小的模型调用参数。第二步就是通过得到的 ALS 训练模型参数然后进行计算得到对应的基于用户的电影相似表以及基于电影的电影相似表。具体实现如下：

第一步模型训练：这一步主要是通过使用 Spark MLlib 库中的 ALS 算法，将加载的评分表先转化为 Spark 能够处理的 DataFrame 格式，然后将其作为训练模型的数据集，最终依据 RMSE 指标调整参数。ALS 调用，`ALS.train(trainData,rank,iterations,lambda)`，其中 `trainData` 固定为评分数数据集，核心参数为 `Rank`（隐特征向量维度，值越大模型表达能力越强但复杂度越高，实验取值从 50 逐步增至 300）、`Iterations`（迭代次数，值越大越精确但越耗时，实验设为 5）和 `Lambda`（正则化系数，值大可防过拟合，值小可提高泛化能力，实验取值从 0.001 逐步增至 1.0）。通过调试发现在本地运行时将 `Rank` 设置为 200，`iterations` 设置为 5，`Lambda` 设置为 0.1 时 RMSE 较小，即此时拟合误差较小（拟合效果和服务器配置相关）。

第二步，使用训练好的 ALS 模型进行矩阵分解得到电影推荐矩阵，具体流程为：

（1）使用 Spark 连接 MongoDB 将对应的用户评分表数据加载到内存中，然后将评分表的格式转化为 Spark 能处理的 DataFrame 格式。

（2）对原始评分数据执行 `map(_._1).distinct()`和 `map(_._2).distinct()`分别提取去重后的用户 ID 集合和电影 ID 集合，构建特征空间基础数据。将原始评分数据转换为 MLlib 标准 Rating 格式 RDD，作为模型训练输入。

（3）调用 `ALS.train()`方法加载预训练模型（`rank=200`,迭代次数=5,正则化系数=0.1），这一步的参数设置在第一步时在训练模型时已经得到了。

（4）将用户评分表（`userId,movieId,rating`）将这个表中的 `userId` 和 `movieId` 做笛卡尔积，生成（`userId,movieId`）元组，主要是为了预测评分表使用，即每个 `userId` 都能记录每个 `movieId` 的预测评分。

（5）通过 `model.predict()`方法对用户-电影笛卡尔积执行全量预测评分计算。对预测结果执行 `filter(_._rating > 0)`过滤无效评分，按用户 ID 进行聚合，生成每个用户的 Top20 推荐列表，封装为 `UserRecs` 对象后，盖写入 MongoDB 的 `UserRecs` 表作为基于用户的



电影推荐表。

（6）从训练好的 ALS 模型中提取电影特征矩阵 `model.productFeatures`，转换为 `DoubleMatrix` 向量格式。

（7）将电影特征矩阵做笛卡尔积，通过计算余弦相似度，过滤相似度阈值（ >0.6 ）后，按电影 ID 聚合相似度列表，生成基于电影的电影相似矩阵 `MovieRecs` 并持久化到 `MongoDB` 中。

基于 ALS 协同过滤推荐的活动图如图 4.10 所示：

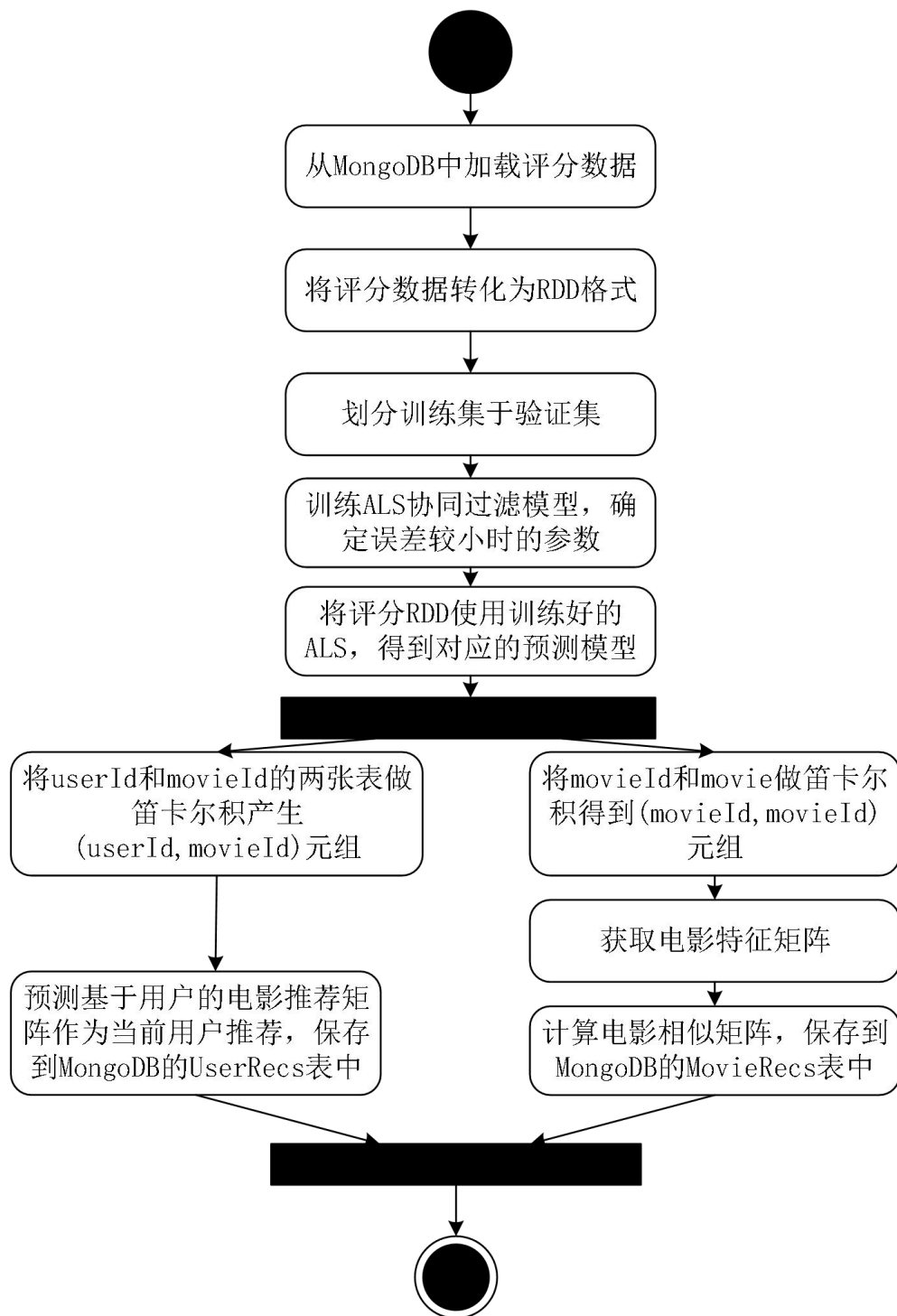


图 4.10 基于 ALS 的推荐算法活动图

4.4 实时推荐模块设计与实现

对于实时推荐算法的讲解在章节 3.3.3 已经讲述，这里不做赘述。实时推荐模块的具体实现细节如下：

实时推荐整体流程：首先用户进行登录之后，选择某个电影进行评分，然后这个评分会通过 Kafka 消息队列将用户评分信息发送给 spark 模块，spark 模块会先从 Redis 中获取用户最近的 K 次评分，然后再到数据库中读取电影相似矩阵然后结合 Kafka 消息队列发送过来的用户评分数据计算出一个实时推荐矩阵，然后用户访问实时推荐接口时后端直接去数据库中查询返回数据。实时推荐时序图如图 4.11 所示：

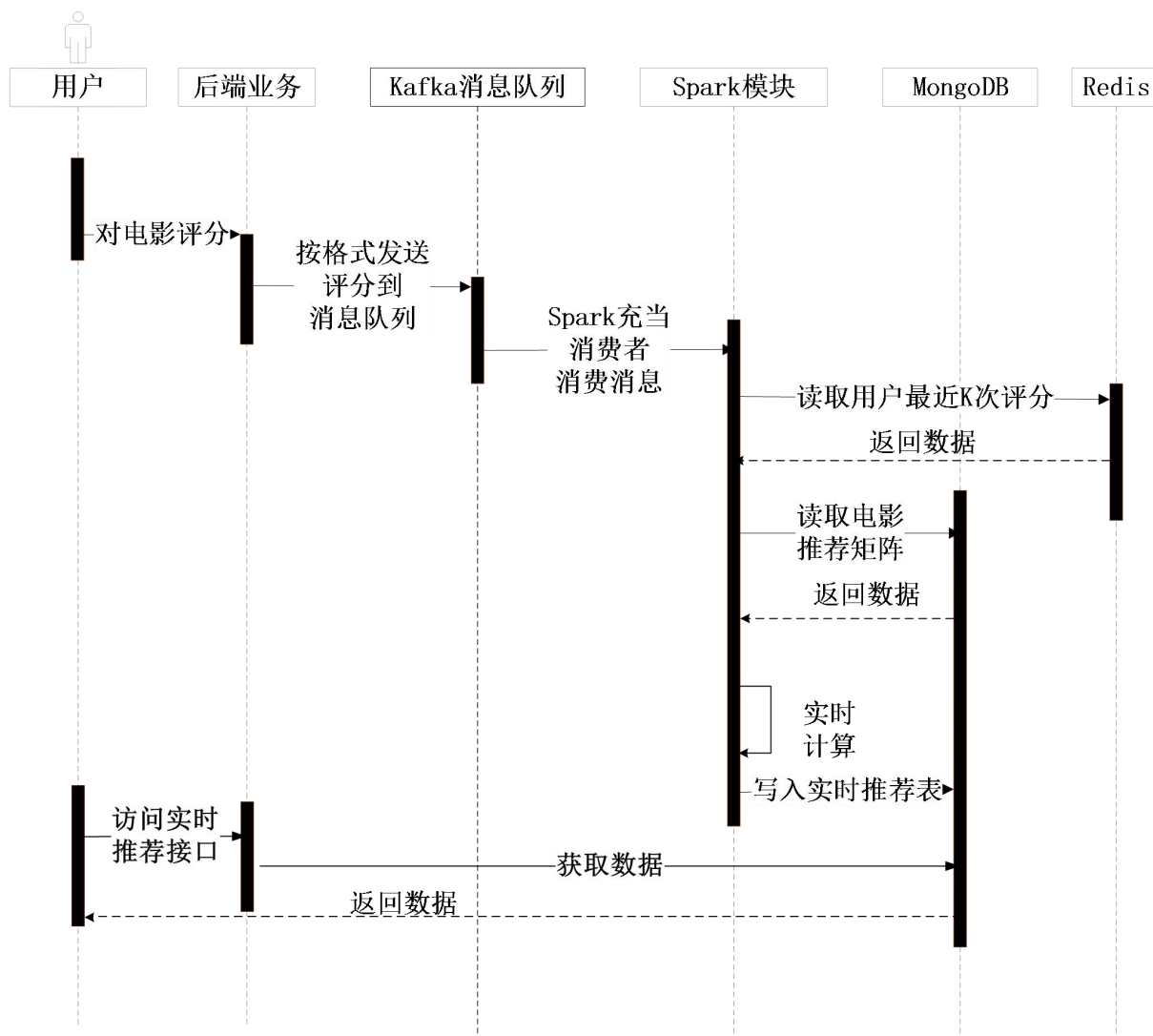


图 4.11 实时推荐模块时序图

系统对应消息队列模型如图 4.12 所示：



图 4.12 系统 Kafka 数据流向图

首先后端主业务充当生产者：当用户对电影进行评分之后，后端业务首先会判断该电影是否评分过，如果没有那么保存评分数据到 **rating** 表中，然后将用户评分保存到 Redis 中的 List 集合中，最后后端业务会充当消息队列生产者按照 UID|MID|SCORE|TIMESTAMP 的格式发送消息到消息队列。

其次使用 Spark 大数据处理模块（充当消息队列消费者）完成实时推荐算法对应实时推荐步骤如下：

（1）系统初始化与数据加载：配置 Spark Streaming 环境，从 MongoDB 加载预计算的电影相似度矩阵并广播至集群，建立 Kafka 消费者连接。

（2）实时评分流处理：通过 Spark Streaming 接收 UID|MID|SCORE|TIMESTAMP 格式评分数据消息，解析为元组(userId, movieId, score, timestamp)。

（3）候选电影生成：根据当前电影 ID 从相似度矩阵获取 Top 20 相似电影，过滤用户已评分电影，按相似度降序保留未评分电影作为候选。

（4）用户近期行为获取：从 Redis 查询用户最近 20 条评分记录。

（5）实时推荐计算：遍历候选电影与用户近期评分记录，按计算电影间相似度，当相似度>0.7 时，结合评分值和增强减弱因子计算最终推荐得分，按电影 ID 分组聚合得分生成推荐列表。

（6）结果存储与更新：删除 MongoDB 中用户原有实时推荐记录，插入新计算结果至 StreamRecs 集合。

（7）系统运行与监控：启动 StreamingContext，保持进程持续运行
系统实时推荐消费者设计流程图如图 4.13 所示：

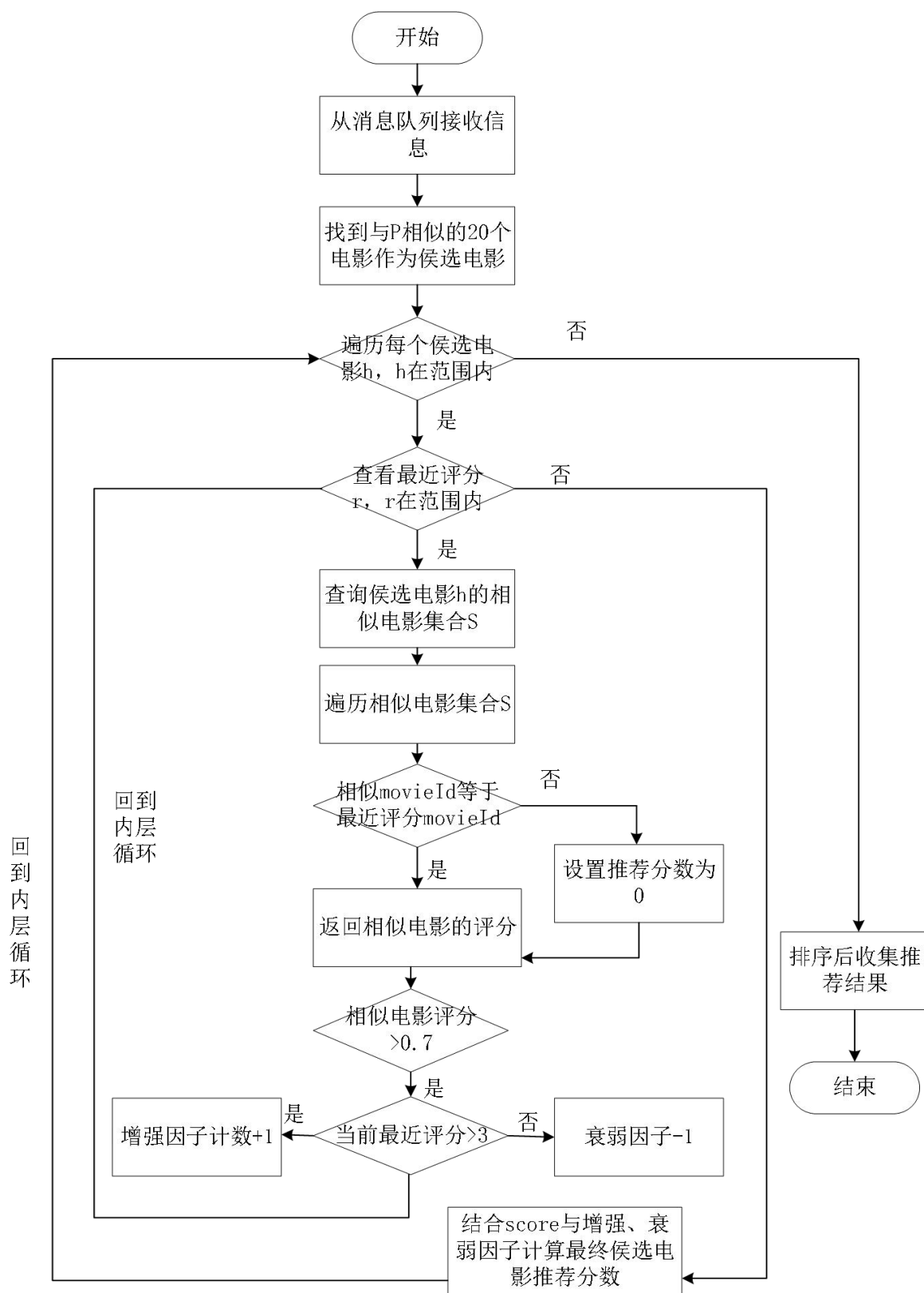


图 4.13 实时推荐消费者处理流程图

第五章 系统模块实现

本系统主要实现了四部分功能，即用户基本业务模块、管理员管理模块以及离线推荐模块和实时推荐模块的实现。本章节对系统各个功能进行了实现。

5.1 基础业务模块实现

首页面介绍，用户点击本系统默认进入系统首页，最开始是一个电影的轮播图，页面最左边和页面最上面作为页面的导航栏。刚开始进入页面用户是处于未登录状态，因此本系统只能提供基本的业务功能，以及只提供统计相关的推荐以及基于电影相似的推荐，个性化推荐复用统计推荐的数据防止页面空白。电影首页如图 5.1 所示：



图 5.1 系统首页

5.1.1 用户登录注册实现

对于新用户能够便捷地完成账号注册流程，而老用户则可直接凭借自身账号与密码登录系统，进而畅享系统所提供的个性化推荐服务。系统的登录以及注册界面如图 5.2 和图 5.3，用户进行登录之后会对用户权限即 `userRole` 字段进行检查，如果是管理员那么会展示管理员管理界面窗口。



图 5.2 登录页面



图 5.3 系统注册界面

5.1.2 电影搜索功能实现

用户可以在系统首页中的搜索框中搜索想要观看的电影，并且支持模糊搜索，同时也支持用户按照电影类别进行搜索，最终系统会返回相关电影信息。搜索界面效果图如图 5.4 所示：

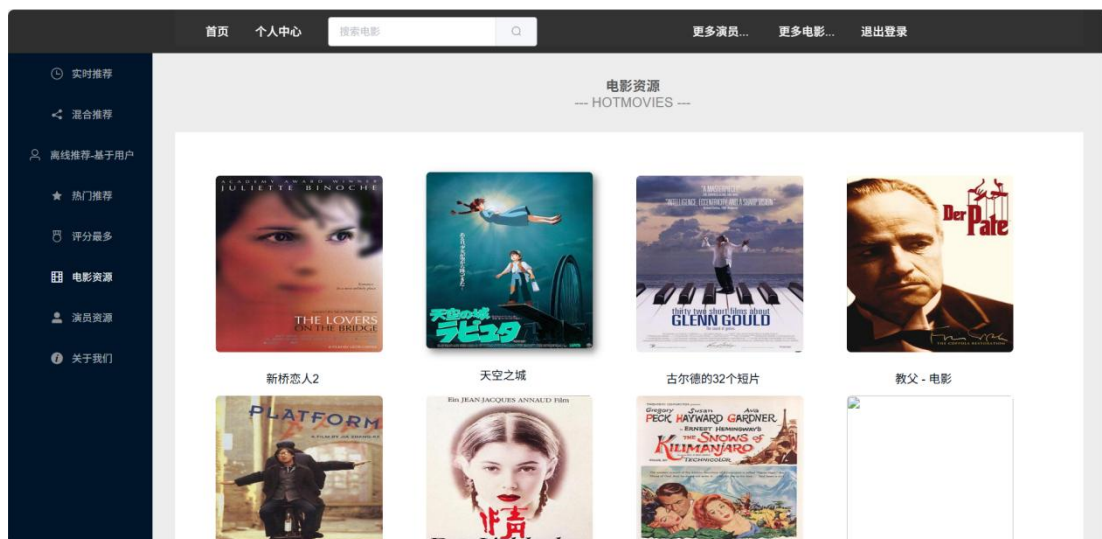


图 5.4 电影搜索功能界面

5.1.3 查看电影详情

电影详情主要是提供给用户查看用户感兴趣的电影的相关信息，用户点击对应的电影图片之后会进行跳转。然后电影详情页面就会展示和该电影相关的内容，比如详情描

述、演员、用户评分、评论等信息实现页面如图 5.5 所示：

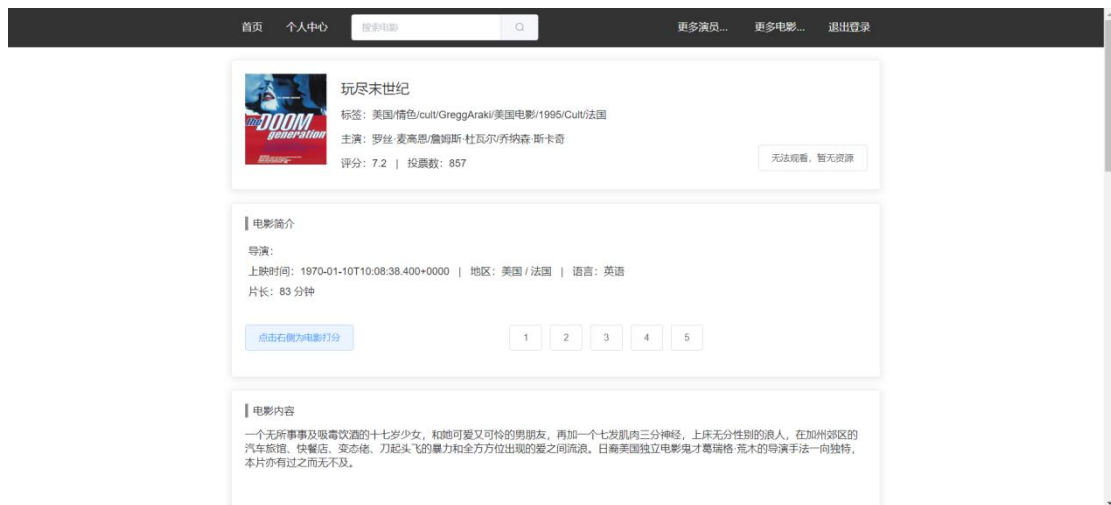


图 5.5 电影详情页面

5.1.4 电影评论功能

用户进入电影详情之后，可以根据查看到该电影的所有用户评论信息，以及自己也可以为该电影进行评论，实现页面如图 5.6 所示：



图 5.6 电影评论功能

5.1.3 演员资源功能实现

用户点击演员资源之后系统会默认展示本系统中收集的相关演员信息，如图 5.7 所示：



图 5.7 演员资源界面

用户如果对演员该远远或者导演等感兴趣可以点击对应的演员或者导演，然后进入主页查看该演员的详细信息，包括演员基本信息以及该演员本系统中拥有的影片资源对应页面如图 5.8 所示，并且前端界面采用滑动查看的方式方便用户使用，同时如果用户对该演员或者导演相关电影感兴趣用户可以点击对应的电影进行查看电影详情。



图 5.8 演员详情

5.1.4 个人中心功能实现

个人中心包括用户个人信息、历史评论、历史评分等功能具体如下：

（1）个人信息

个人信息管理主要用户用户修改自己的相关信息，包括昵称、绑定的手机号、喜爱电影的标签信息以及个人宣言等信息。如图 5.9 所示：



图 5.9 个人信息界面

（2）我的评论

我的评论功能主要为提供用户查看自己评论过的电影信息并提供给用户管理自己评论过的言论，同时支持点击对应评论的电影后可以查看该电影详细信息如图 5.10 所示：



图 5.10 我的评论界面

（3）评分历史

与“我的评论”功能类似，评分历史主要也是为用户提供查看之前评分过的电影集合如图 5.11 所示：

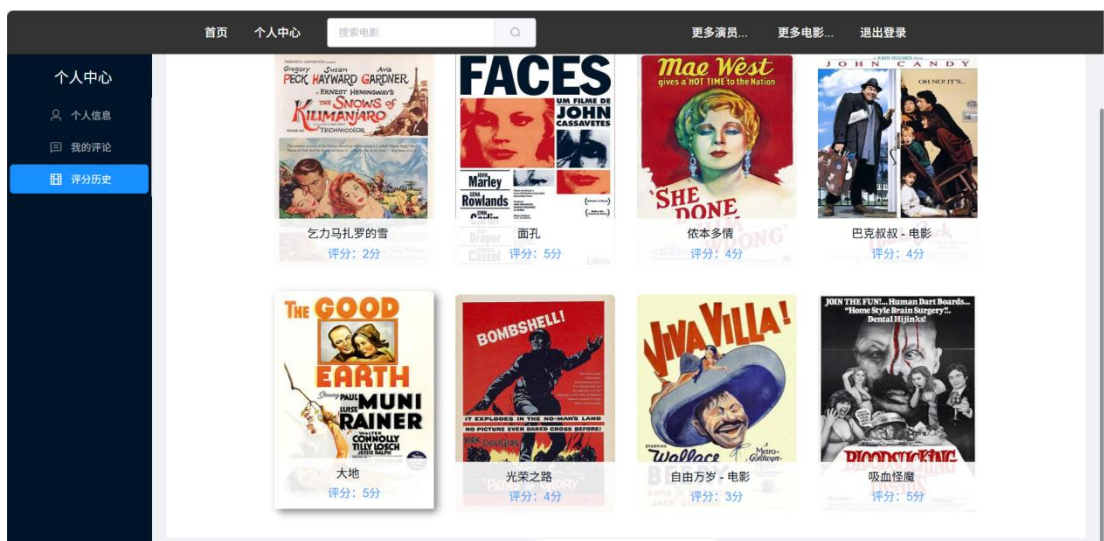


图 5.11 评论历史

5.2 离线推荐模块实现

5.2.1 基于统计的推荐

(1) 热点推荐

热点推荐为推荐每个月评分数量最多的电影即近期最热电影，实现页面如图 5.12 所示：

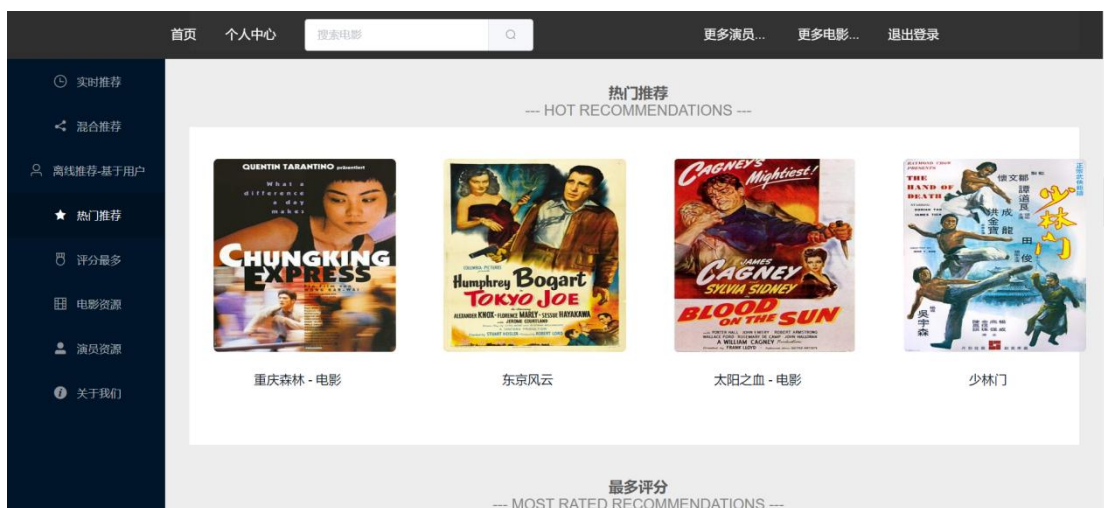


图 5.12 热点推荐页面

(2) 评分最多推荐

评分最多推荐和热点推荐类似，主要区别就是热点推荐主要统计的是评分表 rating 中当月历史数据的评分最多的 TopK 内容然后进行展示，而评分最多推荐采用的是数据集为所有评分历史数据，对应页面如图 5.13 所示：

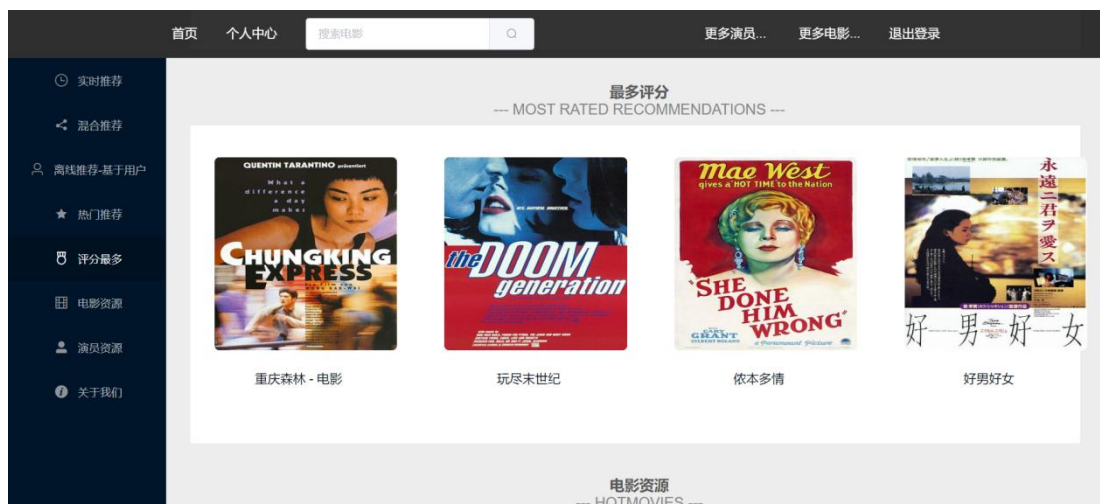


图 5.13 评分最多推荐

5.2.2 基于 ALS 协同过滤的推荐

(1) 基于用户的离线推荐

主界面中的离线推荐-基于用户功能主要是为用户展示基于 ALS 协同过滤算法得到的用户推荐离线表中的推荐数据，即结合用户的历史行为数据找到用户可能感兴趣的电影，实现页面如图 5.14 所示：

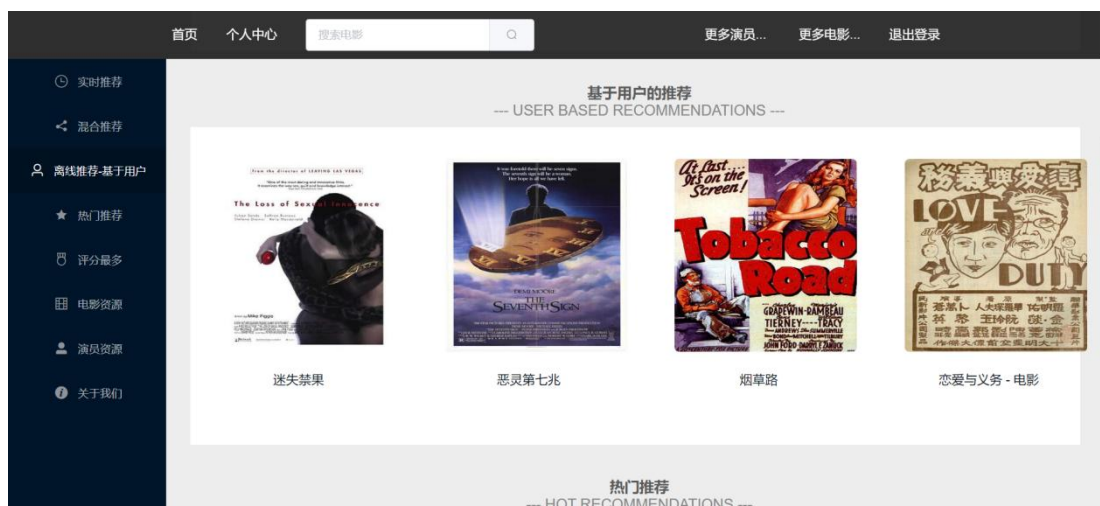


图 5.14 基于用户的离线推荐页面

(2) 基于电影的离线推荐

点击电影详情后电影详情界面里面会展示猜你喜欢，该推荐为基于电影的电影推荐，原理和基于用户类似同样采用的是基于 ALS 的协同过滤算法，计算电影之间的相似度采用的是余弦相似度计算得到，基于电影的离线推荐页面如图 5.15 所示：

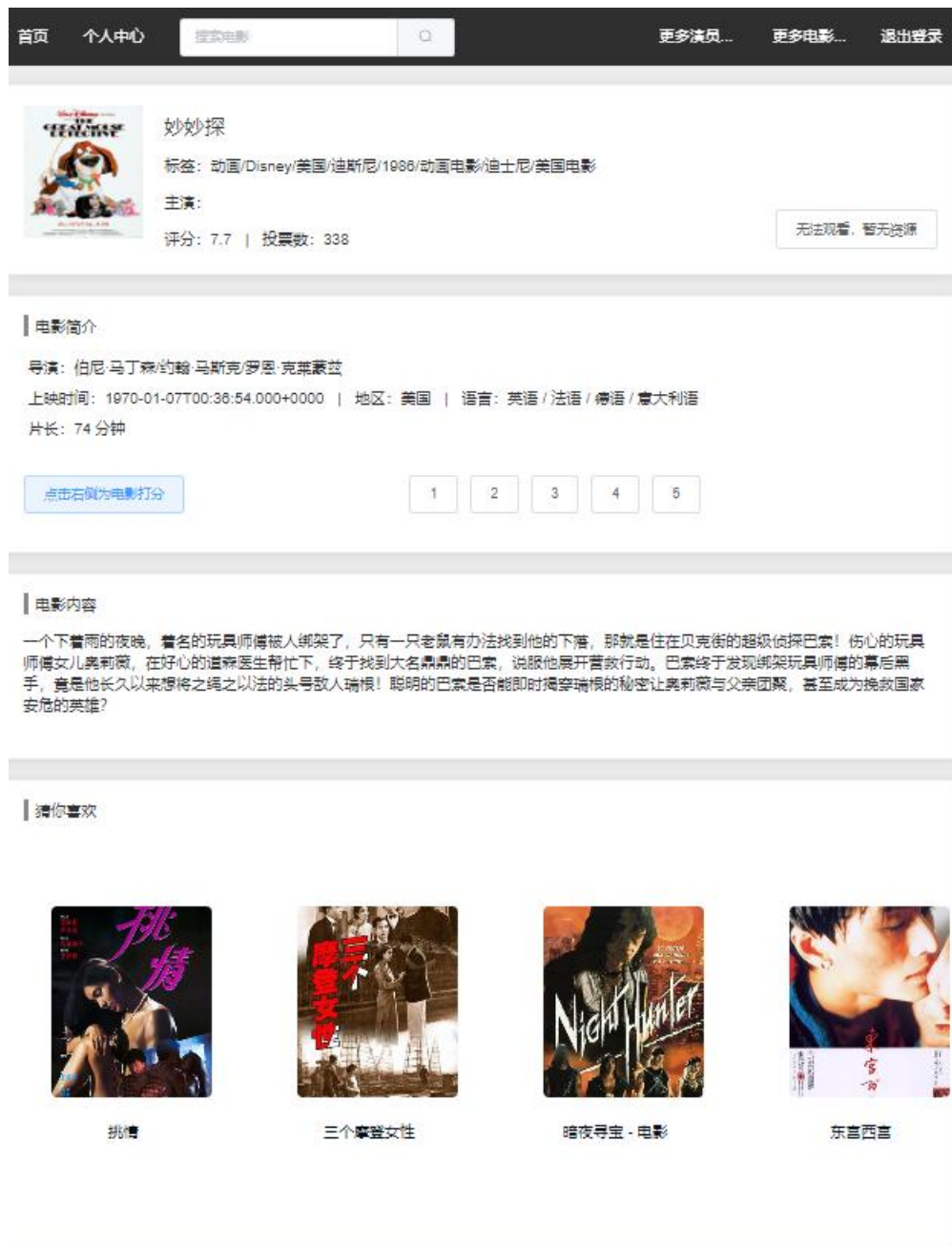


图 5.15 基于电影的离线推荐页面

5.3 实时推荐模块实现

5.3.1 实时推荐实现

当用户在电影详情页面对电影进行打分之后，系统会将用户的打分信息缓存的 Redis 中作为用户近期评分数据，然后将打分电影的相关信息结合本系统的实时推荐算法进行计算最终得到一个新的实时推荐表用于给用户动态展示，达到实时变更的效果。用户打分之前实时推荐页面如图 5.16 所示：

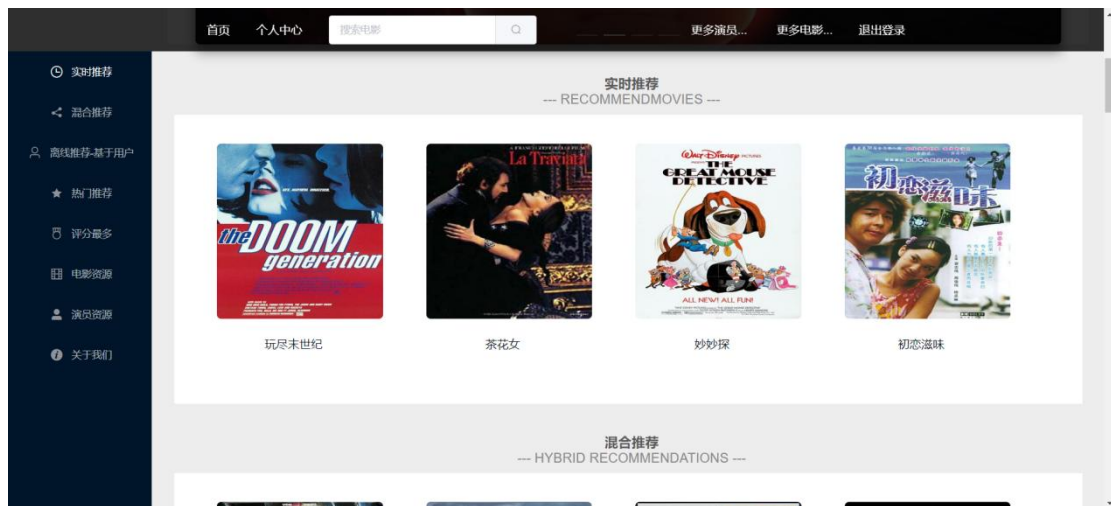


图 5.16 实时推荐页面

用户先点击自己感兴趣的电影进入到电影详情页后，往下滑动可以发现该电影的电影评分选择，之后点击对应的对应的星级后对该电影进行评分，之后评分按钮会变成重新评分按钮支持用户重评分，对应页面如图 5.17 所示：

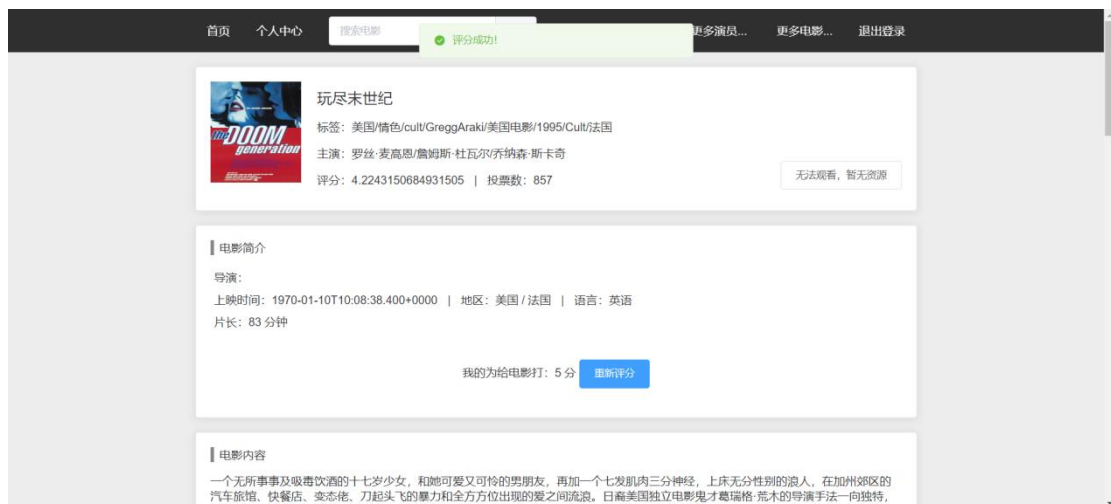


图 5.17 用户打分页面

用户完成评分之后后台会运行实时更新算法进行计算用户实时更新表，然后用户在返回系统首页查看实时推荐或者混合推荐之后就可以看到电影推荐的动态更新了。更新后的页面如图 5.18 所示：

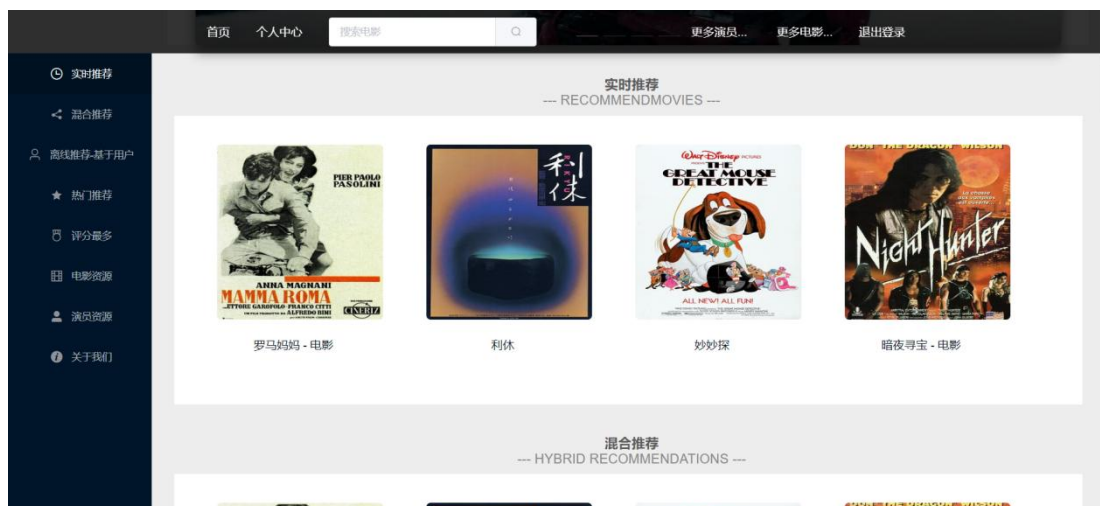


图 5.18 用户打分后实时推荐变化页面

5.3.2 混合推荐实现

混合推荐本系统是采用加权的方式进行推荐即结合了实时推荐表 SreamRecs 表，离线基于用户的电影推荐表以及 Elasticsearch 完成的基于内容的电影推荐混合加权而展示，因为混合推荐对于实时要求较高，所以实时推荐中的相关电影加权值设置的较大为 0.4，而其它两个的推荐分数加权值为 0.3，最终将每个电影的推荐分数乘以加权值的到最终推荐分数，最后排序推荐 top4 的电影。同样和实时推荐类似当用户进行评分之后，混合推荐也会跟着变化，对应页面如图 5.19 所示：

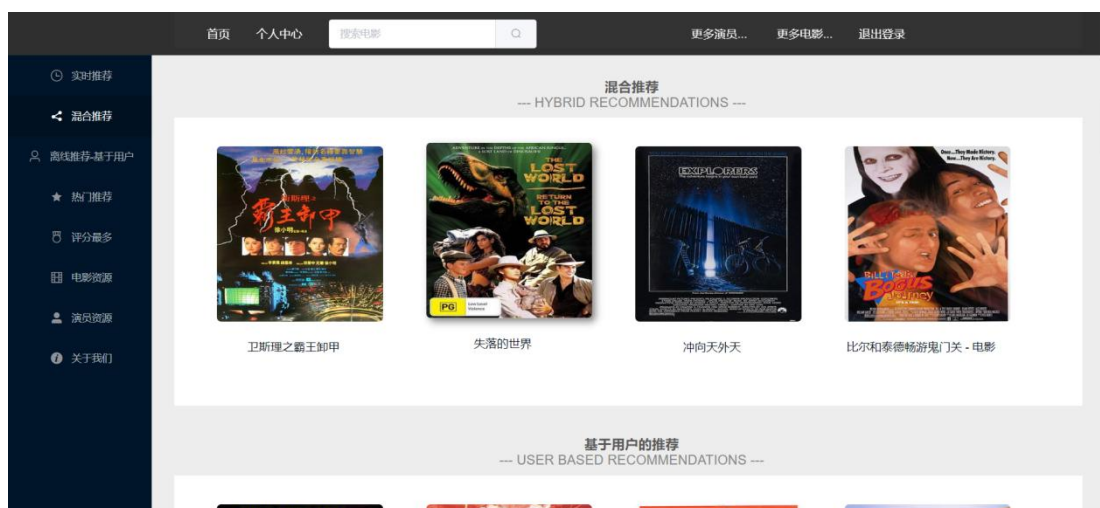


图 5.19 混合推荐页面

5.4 协同过滤冷启动问题

本系统采用的冷启动解决为，再用户注册后登录，首选后台会先检查用户的相关信息查看用户中 `first` 字段是否为 `true`，如果为 `true` 表示该用户是新用户，那么此时系统会向用户弹出一个电影喜好选择框来收集用户兴趣，然后系统后台会根据用户的兴趣标签使用 `Elasticsearch` 中的 `TF-IDF` 算法做基于用户喜好和电影类别的基于内容的电影推荐，即去匹配电影类别 `genres` 中最符合用户喜好的 `top4` 条电影信息返回。此时个性化推荐就默认使用基于内容的推荐，当用户有了相关行为数据之后再进行个性化推荐，为了增强用户体验，本系统实时推荐和离线推荐采用从用户兴趣标签中随机选取一个标签然后去类别最受欢迎表 `GenresTopMovies` 中随机选取一个类别进行展示，然后混合推荐采用完全基于用户所有兴趣和电影类别进行基于内容推荐。冷启动解决页面如图 5.20 所示：

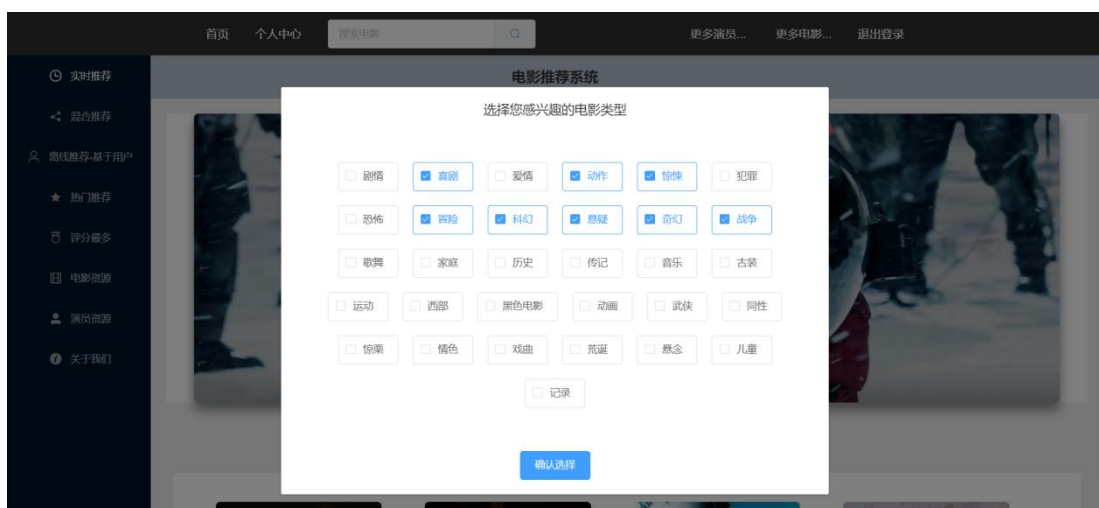


图 5.20 冷启动页面

5.5 管理员业务模块实现

5.5.1 管理员后台首页

管理员后台首页主要为展示本项目的简介以及系统架构和使用的相关技术栈，实现页面如图 5.21 所示：



图 5.21 管理员首页

5.5.2 用户管理实现

为身份为管理员的用户提供用户管理服务, 提供给管理员对用户信息查询功能以及提供对用户的批量删除功能, 实现页面如图 5.22 所示:

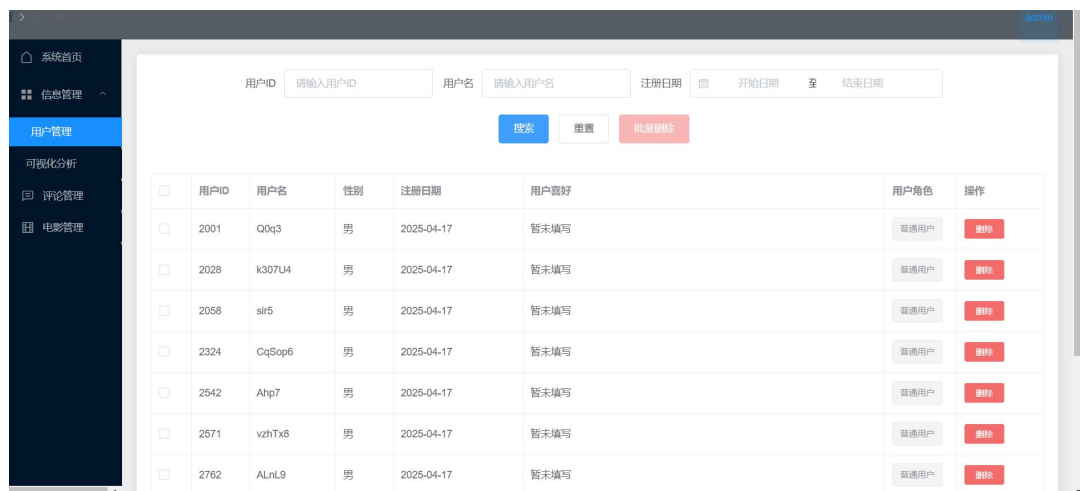


图 5.22 用户管理页面

5.5.3 可视化分析

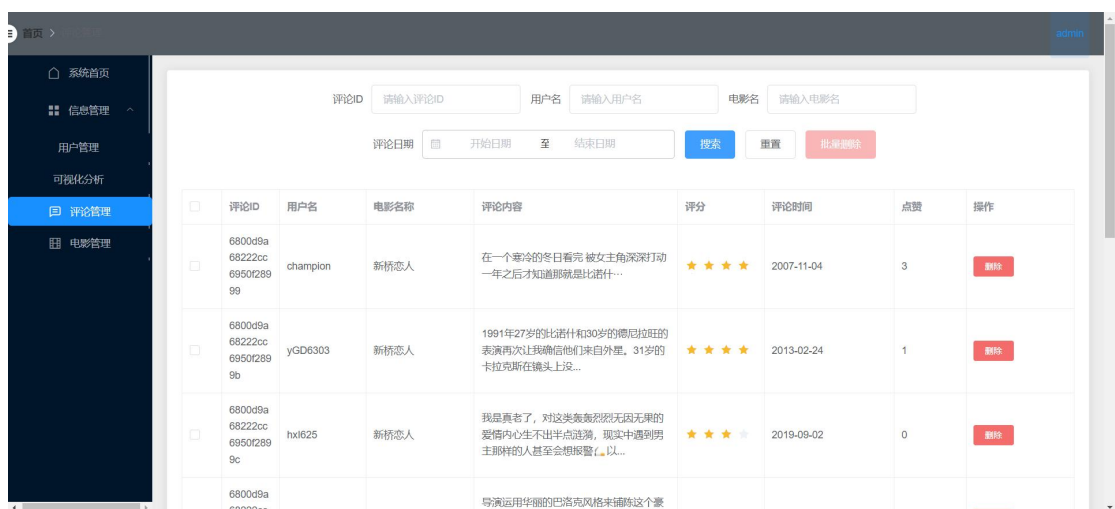
为助力管理员高效决策, 系统将推荐电影的实际推荐转化率以可视化形式呈现, 使管理员能直观掌握数据动态, 从而便于其依据可视化结果及时做出后续优化调整。推荐转化率页面如图 5.23 所示:



图 5.23 推荐转化率可视化页面

5.5.4 评论管理

管理员可对本系统的所有用户评论信息进行按条件查询比如用户名、电影名、评论日期范围等进行查询，通知提供给管理员删除用户评论的功能，方便管理员删除一些用户不当的评论。评论管理页面如图 5.24 所示：



☐	评论ID	用户名	电影名称	评论内容	评分	评论时间	点赞	操作
☐	6800d9a68222cc6950f28999	champion	新桥恋人	在一个寒冷的冬日看完 被女主角深深打动 一年之后才知道那就是比若什...	★★★★	2007-11-04	3	删除
☐	6800d9a68222cc6950f2899b	yGD6303	新桥恋人	1991年27岁的比若什和30岁的德尼拉旺的表演再次让我确信他们来自外星。31岁的卡拉克斯在镜头上没...	★★★★	2013-02-24	1	删除
☐	6800d9a68222cc6950f2899c	hx625	新桥恋人	我是真老了，对这类轰轰烈烈无因无果的爱情内心生不出半点涟漪。现实中遇到男主那样的人甚至会想报警。以...	★★★☆☆	2019-09-02	0	删除
☐	6800d9a68222cc			导演运用华丽的巴洛克风格来描绘这个豪				删除

图 5.24 评论管理页面

5.5.5 电影管理

管理员可对系统电影的新增、修改、上架下架、批量删除等功能，方便系统为用户展示相关电影，实现页面如图 5.25 所示：



推荐管理系统

系统首页
信息管理
用户管理
可视化分析
评论管理
电影管理

电影名称 请输入电影名称 上映日期 开始日期 至 结束日期

搜索 重置 添加电影 批量删除

☐	电影名称	导演	主演	类型	时长(分钟)	上映日期	评分	简介	操作
☐	新桥恋人2	莱奥-卡拉克斯	朱丽叶·比诺什/德尼·拉旺	剧情/爱情	125	1991	8.1	米歇尔...	编辑 下架
☒	天空之城	宫崎骏	田中真弓/横泽启子/初井言荣/寺田农/常田富士男/永井一郎/糸博/鹤屋真知子/神山卓三/安原义人/槐柳二/鹈饲	动画/奇幻/冒险	125	1986	3.458333 3333333 335	古老帝...	编辑 下架
☐	古尔德的32个短片	弗朗索瓦·吉拉德	科鲁姆·费奥瑞	剧情/音乐/传记	98	1993	8.4	Slide #...	编辑 下架
☐			马龙·白兰度/阿尔·帕西诺/詹姆斯·肯恩/理						

图 5.25 电影管理页面



第六章 系统测试

本章节详尽阐述了电影推荐系统的测试流程，全面涵盖功能性、性能、稳定性及安全性等核心测试维度，系统测试分为功能测试与非功能测试两部分：功能测试采用黑盒测试法，此方法无需触及代码层面，却能有效验证系统各项功能是否满足需求；非功能测试则聚焦于评估系统的非功能属性，确保其符合既定要求。

6.1 功能测试

6.1.1 用户功能测试

（1）注册功能测试主要验证功能的正确性以及用户信息的安全性如表 6.1 所示：

表 6.1 注册测试用例表

用例编号	输入/动作	预期结果	实际结果
1	输入用户名和密码	注册成功，跳转登录界面	结果符合预期
2	不输入用户名或密码	无法注册	结果符合预期
3	两次密码输入不一致	注册失败，弹出信息提示	结果符合预期
4	注册已存在的用户名	提示用户名已存在	结果符合预期

（2）登录功能测试如表 6.2 所示：

表 6.2 登录测试用例表

用例编号	输入/动作	预期结果	实际结果
1	输入用户名和密码	登陆成功，跳转系统首页	结果符合预期
2	不输入用户名或密码	提示失败	结果符合预期
3	不输入用户名，不输入密码	提示失败	结果符合预期

（3）电影搜索功能测试如表 6.3 所示：



表 6.3 电影搜索测试用例表

用例编号	输入/动作	预期结果	实际结果
1	输入电影名称	显示搜索的电影	结果符合预期
2	点击电影类别筛选	显示该类型电影列表	结果符合预期
3	点击类别重置	展示所有电影列表分页	结果符合预期
4	输入错误电影名	显示空白内容	结果符合预期
5	输入特殊符号标点	显示空白内容	结果符合预期

（4）电影评分测试如表 6.4 所示：

表 6. 4 电影评分测试用例表

用例编号	输入/动作	预期结果	实际结果
1	点击 1-5 分	提示评分成功，显示已评分数	结果符合预期
2	点击重新评分	提示评分成功，显示重评分数	结果符合预期

（5）电影评论测试如表 6.5 所示：

表 6.5 电影评论测试用例表

用例编号	输入/动作	预期结果	实际结果
1	输入评论信息	提示评论信息，并展示	结果符合预期
2	不输入评论	提示用户可进行评论	结果符合预期

（6）冷启动功能测试如表 6.6 所示：



表 6. 6 冷启动功能测试用例表

用例编号	输入/动作	预期结果	实际结果
1	使用新账号登录系统	弹出冷启动页面， 勾选用户兴趣后 跳转主页	结果符合预期

（7）个人信息修改功能测试如表 6.7 所示：

表 6. 7 个人信息修改功能测试用例表

用例编号	输入/动作	预期结果	实际结果
1	点击编辑资料	弹出编辑界面，并 回显用户基本信 息	结果符合预期
2	输入修改相关资料	提示修改成功	结果符合预期

（8）推荐功能测试如表 6.8 所示：

表 6. 8 推荐功能测试用例表

测试功能	输入/动作	预期结果	实际结果
热门电影推荐	点击热门推荐	显示相关电影	结果符合预期
评分最多电影推荐	点击评分最多推荐	显示相关电影	结果符合预期
离线推荐-基于用户	点击离线推荐-基于用户	显示相关电影	结果符合预期
离线推荐-猜你喜欢	点击电影详情	显示和该电影相似的其他电影	结果符合预期
热门电影推荐	点击热门推荐	显示相关电影	结果符合预期

（9）实时推荐功能测试如表 6.9 所示：



表 6. 9 实时推荐功能测试用例表

测试功能	输入/动作	预期结果	实际结果
实时推荐	对电影评分	评分后，实时推荐 结果发生相应变 化	结果符合预期
混合推荐	对电影评分	评分后，混合结果 发生相应变化	结果符合预期

6.1.2 管理员功能测试

（1）用户管理/评论管理功能测试测试如表 6.10 所示：

表 6. 10 用户管理功能测试测试用例表

用例编号	输入/动作	预期结果	实际结果
1	输入用户 ID 或用户名或者 注册日期范围	筛选出对应的用 户信息进行展示	结果符合预期
2	点击重置	展示所有用户列 表	结果符合预期
3	输入筛选条件有误后者系 统无此信息	展示空白列表	结果符合预期
4	选择勾选单个或多个用户 后点击删除	提示删除成功	结果符合预期
5	勾选的用户身份同为管理 员，然后删除	提示权限不足	结果符合预期

（2）评论管理功能测试如表 6.11 所示：

表 6. 11 评论管理功能测试用例表

用例编号	输入/动作	预期结果	实际结果
1	输入评论 ID、用户名、电 影名或者评论日期范围	筛选出对应的评 论信息进行展示	结果符合预期
2	点击重置	展示所有评论	结果符合预期



续表 6. 11 评论管理功能测试用例表

用例编号	输入/动作	预期结果	实际结果
3	输入筛选条件有误后者系统无此信息	展示空白列表	结果符合预期
4	选择勾选单个或多个评论后点击删除	提示删除成功	结果符合预期

(3) 电影管理功能测试如表 6.12 所示：

表 6. 12 电影管理功能测试用例表

用例编号	输入/动作	预期结果	实际结果
1	输入电影名或者上映日期范围	筛选出对应的电影信息进行展示	结果符合预期
2	点击重置	展示所有电影列表	结果符合预期
3	输入筛选条件有误或系统无此信息	展示空白列表	结果符合预期
4	选择电影进行上架或/下架	提示操作成功并显示电影状态为上映中/下架	结果符合预期
5	点击添加电影，输入电影信息	显示电影添加成功，并展示电影信息	结果符合预期
6	点击编辑电影后输入修改内容	弹出电影编辑框，并回显电影信息	结果符合预期



6.2 非功能性测试

本次非功能测试主要为了测试本系统的推荐功能相关的性能测试以及对于用户信息安全方面的测试。

（1）使用 Jmeter 工具对系统推荐功能进行性能的压力测试，性能测试用例如表表 6.13 所示：

表 6. 13 推荐功能性能测试表

动作	并发测试次数（次/s）	期望响应时（s）	平均响应时（s）
热门电影	1000	<1	<0.65
评分最多	1000	<1	<0.68
基于用户离线推荐	1000	<1	<0.72
基于电影离线推荐	1000	<1	<0.66
实时推荐	1000	<1	<0.82
混合推荐	1000	<1	<0.88

（2）对系统的安全性进行测试，安全测试用例如表 6.14 所示：

表 6. 14 安全测试用例表

用例编号	输入/动作	预期结果	实际结果
1	查看数据库用户密码	用户密码被加密	结果符合预期



第七章 总结与展望

7.1 总结

当下，电影产业产值持续上扬用户观影需求也日益提高。单一传统推荐法难合用户心意，且数据维护量大，传统前后端开发框架已难以适配该应用场景。本文针对此问题，本文根据用户需求采用了多种推荐方式进行结合以适应用户需求，同时采用基于 ALS 的协同过滤推荐算法解决了传统的基于邻近的协同过滤算法的稀疏矩阵带来的问题，采用基于内容的推荐算法弥补协同过滤算法的冷启动问题。对于实使推荐部分引入增强因子和衰弱因子增强用户近期的行为权重，对基于 ALS 的离线推荐进行改进增强用户的体验。对于系统大数据量的计算本系统采用分布式计算框架 Spark 完成提高系统的计算效率。本文从分析用户需求开始，构建了推荐系统的整体架构。在架构设计中规划了多样化的用户功能与角色权限，旨在确保系统能够精准适配各类用户的个性化需求；同时还深入设计了软件系统的架构，并详细阐述了软件的工作原理，以此保障系统具备技术灵活性与可扩展性。

此外，在技术层面，本系统采用分模块开发增强系统的扩展性和稳定性。在数据库设计环节，本系统结合业务需求建了数据库表实体树立了各数据表之间的内在联系，最终基于 MongoDB 数据库完成了高效且合理的数据库架构设计。在后端实现时本系统选用了 SpringBoot 框架进行开发基于 MVC 架构模式进行开发实现高扩展的系统业务逻辑。考虑到推荐系统用户行为数据较大传统的后端开发不能很好的应对，因此本系统引入了 Spark 生态进行数据处理。最终完成系统实现后，同步开展功能性与非功能性测试，结果显示系统各项功能运行正常，且交互响应速度达秒级标准。

7.2 展望

随着互联网技术和电影文化的发展，电影推荐系统仍有广阔提升空间。技术上，可进一步优化混合推荐算法，融合更多前沿算法提升推荐精准度；持续优化分布式计算框架 Spark 的参数配置，提升大数据量处理效率。功能上，深入挖掘用户潜在需求，增加个性化定制维度。交互层面，结合人工智能语音、图像识别等技术，打造更自然便捷的交互体验。同时，强化系统安全防护，保障用户数据隐私，为用户提供更优质、高效、安全的电影推荐服务。



参考文献

- [1] 李希.基于深度学习的电影推荐系统的设计与实现[D].电子科技大学,2023.DOI:10.27005/d.cnki.gdzku.2023.001020.
- [2] 徐必成.基于混合方法的电影推荐系统的研究与实现[D].湖南大学,2022.DOI:10.27135/d.cnki.ghudu.2022.003890.
- [3] 张宁.基于 Spark 的电影推荐系统设计与实现[D].西安电子科技大学,2023.DOI:10.27389/d.cnki.gxadu.2023.003276.
- [4] 尹建凯.基于深度学习和协同过滤的电影推荐系统设计与实现[D].华东师范大学,2024.DOI:10.27149/d.cnki.ghdsu.2024.003123.
- [5] 潘悦.基于内容与协同过滤算法的电影推荐系统研究[D].黑龙江大学,2023.DOI:10.27123/d.cnki.ghlju.2023.002017.
- [6] 姜凌峰.基于协同过滤和深度学习的混合推荐算法研究与应用[D].东华大学,2022.DOI:10.27012/d.cnki.gdhuu.2022.001073.
- [7] 潘韵文.一种改进的 ALS 模型协同过滤推荐算法的研究[D].大连交通大学,2023.DOI:10.26990/d.cnki.gsltc.2023.000311.
- [8] 徐佳蕾.智能推荐系统在新媒体平台中的应用[J].卫星电视与宽带多媒体,2024,21(24):49-51.
- [9] 顾明星,张梦甜.结合改进用户聚类与 LFM 模型的协同过滤推荐算法[J].科技与创新,2023,(17):18-21+24.DOI:10.15913/j.cnki.kjycx.2023.17.005.
- [10] 孔欢.基于隐语义模型推荐算法研究[D].江苏科技大学,2021.DOI:10.27171/d.cnki.ghdcc.2021.000572.
- [11] 王璇,杜宇超,杜军,等.基于动态矩阵分解模型的电影推荐系统研究[J].电子器件,2022,45(02):483-489.
- [12] 张金霜.基于 TF-IDF 的微博特征提取网络舆情分析系统设计[J].电子技术,2025,54(02):59-61.
- [13] 杨运强.基于 Spark 大数据的智能推荐系统设计与实现[J].信息记录材料,2025,26(02):150-152.DOI:10.16009/j.cnki.cn13-1295/tq.2025.02.041.
- [14] 王志涛.基于 B/S 模式的项目管理信息系统开发与设计[J].办公自动化,2024,29(24):84-86.
- [15] 林娴斐.基于时间因子的分布式协同过滤系统研究[D].北京理工大学,2016.
- [16] 卓雪虹,任璐圆.一种基于信令大数据分析的网络故障发现方法[J].通讯世界,2020,27(03):46-47.



致谢

随着本论文的收尾，我的大学生涯也进入了尾声。回望大学四年，满是成长印记。出于对计算机领域的热爱，大一期间我努力学习争取到了转专业到计算机的机会，自此踏上挑战之路。面对全新的课程体系与复杂知识，我日夜钻研，从基础编程到算法设计，不断填补知识空白，磨炼出坚韧不拔的毅力。为积累实践经验，我主动出击寻找实习，在项目中将所学化为所用，真正做到将输入转化为输出，这也增加我的社会经历。本科期间，我以高标准要求自己，各学科成绩优异。这一路，我凭借热爱、坚持与拼搏，完成了蜕变。

首先，我要衷心感谢我的学业指导和论文指导老师。从论文的选题、构思到撰写、修改，每一个环节都离不开导师的悉心指导。导师严谨的治学态度、深厚的学术造诣和无私的奉献精神，让我深受感染和鼓舞。每当我遇到难题时，导师总是耐心地为我解答，给予我宝贵的建议和启发，让我能够顺利地完成论文。除此之外导师在我的大学生涯中还扮演者我的人生规划导师，让我在大二的时候就能确定好自己以后的方向，在老师的规划和指导下自己也早早的确认了自己以后的从业方向，让我有了奋斗的目标。

其次，我要感谢学院的各位任课老师。是你们用渊博的知识和生动的教学方法，为我打开了专业知识的大门，让我在学术的道路上不断探索和进步。你们的教诲如明灯，照亮了我前行的方向。以及我要感谢我的辅导员张潇月老师，在我大二转专业到计算机之后老师就对我的学习以及生活上给予了很大的帮助。同时还要感谢和伴我左右的同学和朋友，你们的帮助为我的大学生涯添加了很多的阳光。

最后，我满心感激地想要对我的家人说声谢谢，感谢你们一直以来给予我细致入微的关怀与疼爱。每当我站在人生的十字路口，面临艰难抉择时，你们总会耐心地为我出谋划策，给出切实可行的建议；即便我偶尔做出一些让你们难以理解的决定，你们依然毫无保留地给予我支持与鼓励，这份深情厚谊我将永远铭记于心。

毕业在即，我深知这既是一个终点，也是一个新的起点。我将带着这份感恩之心，踏上新的征程，努力前行，不辜负大家的期望。